Software-as-a-Graph: Heterogeneous Graph Learning for Pre-Deployment Dependability Analysis of Complex Distributed Systems

Abstract

Pre-deployment dependability assessment of asynchronous distributed systems is hindered by absent runtime telemetry and static code analysis's blindness to communication topology. This paper presents Software-as-a-Graph (SaG), a static system analysis framework that builds typed multigraphs from Architecture-as-Code manifests over five entity types, and pairs two parameter-independent pathways: a relation-specific Heterogeneous Graph Transformer (HGT) with Quality-of-Service (QoS) edge encodings that forecasts cascading failure blast radii, and an interpretable ISO/IEC 25010 layer attributing fragility to Availability (single points of failure) or Fault Tolerance (cascade hubs) to prescribe targeted repairs.

Across seven synthetic scenarios and three open-source reference systems (Autoware.universe ROS 2, GCP Online Boutique, Train-Ticket): (1) under distribution shift, relation typing helps at matched QoS encoding — $\rho = 0.439$ vs. 0.086 without QoS edge features, 0.608 vs. 0.363 with them (8/8 folds, $p = 0.0078$) — whereas in-distribution the typed and untyped models are indistinguishable, making typing a generalization property rather than a fitting advantage; (2) against a training-free QoS-weighted centrality baseline the learned model is matched, not beaten, on ranking (+0.037, $p = 0.64$) and on critical sets ($F_1@K = 0.414$ vs. 0.380; 4/8 folds, $p = 0.47$); (3) SaG transfers zero-shot to the reference systems ($\rho = 0.685$–0.778), though its own release gate passes on none of them; (4) an HGT forward pass costs 43.7 ms at 2,000 components, while the complete gate, dominated by graph analysis, runs in 0.02–27.4 s. The case for training a model rests on relational attention, per-relationship criticality and standards-grounded attribution, not on ranking accuracy.

Keywords: Graph representation learning, Heterogeneous graph neural networks, Distributed systems dependability, Cascading failures, Static system analysis, Explainable AI

1. Introduction

1.1. Motivation

Modern large-scale distributed software systems increasingly rely on asynchronous, event-driven, and publish–subscribe (pub-sub) architectures. Across diverse domains—from autonomous driving (ROS 2 [1]) and enterprise event streams (Apache Kafka [2]) to cyber-physical backbones (DDS [3]), IoT fleets (MQTT [4]), cloud-native microservices [5, 6], and distributed AI/LLM serving clusters—pub-sub decouples producers and consumers in space, time, and synchronization [7]. Components interact indirectly through intermediate message topics and brokers without maintaining direct static references. Furthermore, modern middleware specifications allow engineers to configure deployment-time Quality-of-Service (QoS) policies—such as reliability guarantees, durability, message priorities, and delivery deadlines—to govern how traffic behaves under peak load and network stress.

While this architectural decoupling confers elastic scalability and operational flexibility, it creates a formidable **visibility barrier** for system performance, reliability, and computational sustainability:

- **Indirect Failure and Degradation Pathways:** In traditional synchronous architectures (e.g., RESTful HTTP or gRPC), component interactions follow explicit caller–callee invocation paths. In asynchronous pub-sub and event meshes, publishers and subscribers share no direct references. Cascading failures, queue head-of-line blocking, and backpressure propagate across hidden logical paths spanning brokers, shared topics, colocated execution nodes, and shared libraries.

- **Distinct Degradation Mechanisms:** Disturbances in complex distributed systems do not propagate in a uniform manner. They manifest either as *sequential cascades* (e.g., a slow subscriber causing broker queue saturation and upstream backpressure) or as *simultaneous blast radii* (e.g., a shared runtime library crash, memory exhaustion, or host machine outage instantly disabling multiple colocated services). Conventional architectural diagrams and static call graphs fail to represent these multi-layer dependencies.

Addressing these architectural vulnerabilities is most effective and cost-efficient **prior to deployment**, during design and Continuous Integration / Continuous Delivery (CI/CD), adhering to established foundational principles of dependable computing [8]. However, at design and build time, **no runtime telemetry, distributed tracing, or operational logs exist**. Consequently, software architects, performance engineers, and Site Reliability Engineers (SREs) face two fundamental questions without operational data:

1. *Which components, message topics, and communication links are systemically critical to system dependability and performance?*
2. *Why are they critical, and what specific architectural repair (such as replicating a message broker, decoupling an over-subscribed topic, or sandboxing a shared library) will most effectively eliminate that risk?*

The same questions matter for performance and for computational cost: cascading failures that reach production drive restart loops, retry storms and failover thrashing that consume capacity without completing work. Design-time hardening is the cheapest point at which to remove those defects. We flag at the outset that this motivation is broader than what this paper measures — we predict failure impact and report analysis cost, and we neither predict a latency or throughput quantity nor measure energy (Sections 7.5 and 8.2).

1.2. Problem Statement: The Architecture–Code Gap and the Black-Box AI Challenge

We formulate pre-deployment dependability and performance analysis around two distinct, complementary tasks:

1. **Failure-Impact Forecasting (Predictive Pathway) — the primary task.** We forecast dynamic cascading failure blast radii and identify critical components using a data-driven, relation-specific model over learned topological representations. Closed-form topological metrics capture broad connectivity cheaply; whether they also resolve multi-hop, relation-dependent cascade spread across heterogeneous channels is an empirical question, which we test directly against such a baseline (Section 7.1). Our predictive pathway is trained and evaluated against independent simulation ground truth as a ranking and critical-set identification model.
2. **Explainable Criticality Attribution (Explanation Layer) — what a rank alone cannot say.** A ranked shortlist indicates *where* risk lies, but not *how to fix it*. We therefore pair the predictor with an interpretable structural quality profile grounded in ISO/IEC 25010 [9] and ISO/IEC 25019 [10]. This layer diagnoses the *qualitative root cause* of vulnerability—distinguishing, for instance, an unreplicated single point of failure from a high-coupling maintainability bottleneck—to guide concrete repairs. It serves strictly as an attribution model, not a ranking model.

This separation is architectural rather than merely presentational: both pathways operate on the same graph but share no parameters, and neither is trained on the other's output. The coupling term that could connect them is disabled by default and reported only as an ablation (Section 4.2). Maintaining this independence allows SaG to identify components that are structurally central yet operationally low-impact—a nuanced diagnosis unattainable by either pathway alone.

Existing software engineering approaches fail to bridge what we define as the **Architecture–Code Gap**: *a distributed system can have pristine, bug-free source code within each individual service, yet remain fragile to catastrophic global outages caused by hidden architectural single points of failure (SPOFs) or mismatched middleware Quality-of-Service (QoS) contracts.* Although classical architecture evaluation methods such as the Architecture Tradeoff Analysis Method (ATAM) [11, 12] identify architectural risks, and software studies

analyze architectural technical debt [13] and bad smells [14], they rely on manual stakeholder elicitation rather than quantitative structural learning. Prevailing automated paradigms each leave part of this gap unaddressed. Static code analysis [15, 16, 17, 18] inspects complexity, cohesion and coupling *within* a service but cannot observe message queues or cross-host propagation. Runtime chaos engineering [19] injects real faults but needs a provisioned cluster, carries operational risk, and arrives after the architecture is fixed. Homogeneous centrality metrics [20, 21, 22, 23] flatten the system into an untyped graph in which a message topic, a shared library and an execution host are indistinguishable. Section 2 develops each of these in turn.

Furthermore, while machine learning has demonstrated remarkable success across software engineering, contemporary AI approaches applied to system dependability often function as uninterpretable black boxes. Deep neural models frequently output scalar risk scores or latent embeddings without providing transparent, actionable rationales for their predictions. In mission-critical software engineering, an opaque risk score is inadequate: developers and SREs cannot refactor code or reconfigure infrastructure without understanding *why* a component is vulnerable and *which* architectural mechanism is compromised.

1.3. The Software-as-a-Graph (SaG) Approach

To bridge the Architecture–Code Gap while overcoming the black-box AI challenge, this work introduces **Software-as-a-Graph (SaG)**, an AI-driven pre-deployment **Static System Analysis (SSA)** framework. SaG ingests Architecture-as-Code manifests and executes a four-stage pipeline:

1. **Typed Multigraph Formulation:** SaG models the distributed architecture as a typed, directed multigraph over five core entity types: Applications, Brokers, Topics, Execution Nodes, and Shared Libraries (Section 3.1).
2. **QoS-Aware Logical Dependency Projection:** Using six formal projection rules, SaG derives a semantic `DEPENDS_ON` dependency layer that captures both sequential cascades (via topics and brokers) and simultaneous blast radii (via shared libraries and node colocation), weighted by declared QoS contracts (Section 3.2).
3. **Heterogeneous Graph Learning for Failure Forecasting (Predictive Pathway):** SaG trains a **Heterogeneous Graph Transformer (HGT)** whose relation-specific attention lets a `USES` edge into a shared library propagate differently from a `PUBLISHES_TO` edge into a topic. It forecasts cascading blast radii, ranks critical components, and outputs per-relationship criticality alongside multi-task quality outputs (Section 4).
4. **Explainable Quality Attribution (Explanation Layer):** To explain *why* a flagged component is critical, SaG combines code-level SCA metrics with topological properties into a deterministic **Reliability–Maintainability (RM)** attribution model (Section 5). Reliability decomposes into **Fault Tolerance** (error propagation depth) and **Availability** (single-point-of-failure exposure), pointing to distinct repairs. Because it is a linear, propagation-free aggregate by design, it explains *why* a component is vulnerable rather than how far a cascade travels; its standalone rank correlation is correspondingly modest (Section 7.1).

To ensure methodological rigor, SaG enforces a strict **input–label independence guarantee**: learned models and attribution baselines operate exclusively on the analytical graph G_{analysis}, while ground-truth failure impacts are generated by independent discrete-event simulators operating on the raw structural topology $G_{\text{structural}}$ (Section 4.4).

Figure 1 shows how the two pathways relate. The predictive pathway is the primary one and the only pathway validated against the simulation oracle — the oracle scores rankings, which a quality profile is not — and that oracle is strictly an offline training-and-validation component, never a dependency of online inference. The explanation layer then characterises what the predictor flagged and the remediation that implies. The single link between them is triage rather than data flow: the architect applies the explanation to whatever the predictor ranked. The remediation guidance closes a loop of its own, in which each candidate edit is re-simulated on its own mutated copy of $G_{\text{structural}}$ and kept only if it beats the simulator's seed-to-seed noise.

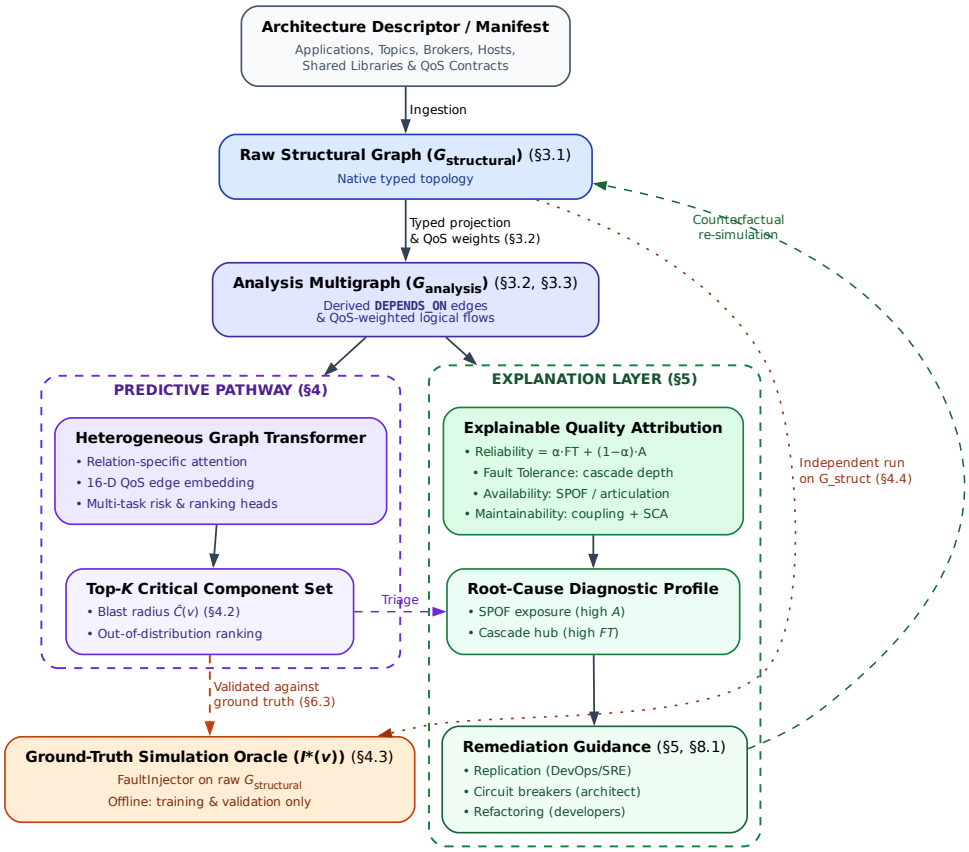

Figure 1: End-to-end architecture of the SaG framework. A shared front end (manifest ingestion → typed multigraph → QoS-weighted `DEPENDS_ON` projection → typed node features) feeds two pathways that share no parameters: the predictive pathway (Section 4), which emits a ranked critical set and per-relationship criticality, and the explanation layer (Section 5), which emits a standards-grounded quality profile. The simulation oracle scores only the former and runs on $G_{\text{structural}}$ alone.

Rationale for Graph Learning vs. Direct Simulation. Since discrete-event simulation $I^*(v)$ defines ground-truth criticality here, it is fair to ask why train a graph model at all rather than run simulation sweeps or closed-form heuristics. Four practical reasons motivate the design, and Section 7.1 tests the last of them adversarially. Simulators inject faults only into active application processes, leaving 30–47% of components per system (topics, hosts) without direct labels, whereas message passing generalizes across labelled and unlabelled entities alike. Cascade simulation is stochastic and seed-sensitive (label standard deviation reaches 0.416), while a trained model learns a smooth, threshold-marginalized surrogate that re-scores an already-analysed architecture cheaply. Neither a simulator nor an unaugmented GNN returns a root cause in standardized quality terms: knowing that a subscriber lost its feed does not distinguish an unreplicated single point of failure from an error-cascade hub, and the two demand different repairs. Finally, dynamic simulators need runnable containers or communication harnesses, whereas graph learning scores Architecture-as-Code manifests before any runtime infrastructure exists. Whether these motivations are borne out empirically is a separate question, and Section 7.1 answers it only partly in the framework's favour.

1.4. Research Questions

This empirical study investigates five research questions:

RQ1 (Predictive Efficacy): *How accurately does heterogeneous graph learning predict cascading failure impact and identify the critical component set, compared with traditional, non-learning network metrics?*

RQ2 (Value of Architectural Typing): *Does modeling distinct entity and dependency types (applications, topics, brokers, hosts, and libraries) yield better failure predictions than homogeneous graph models, and does that advantage hold on architectures the model has never seen?*

RQ3 (QoS Encoding, Calibration and Sensitivity): *How do middleware Quality-of-Service policies, the declared weighting constants of the explanation layer, the choice of simulation oracle, and propagation-threshold and normalization settings affect forecasting performance, stability, and explainability?*

RQ4 (Real-World Generalization): *How effectively does the framework transfer zero-shot to authentic, real-world distributed systems across autonomous driving (ROS 2) and cloud-native microservice architectures?*

RQ5 (Analysis Cost and Computational Sustainability): *What does pre-deployment analysis cost at CI/CD time, which pipeline stage dominates that computational footprint, and does the resulting budget enable sustainable, per-commit architectural gating?*

1.5. Key Contributions

This paper presents four principal contributions:

1. **Heterogeneous Graph Learning for Pre-Deployment Dependability:** A relation-specific Heterogeneous Graph Transformer that forecasts cascading blast radii from Architecture-as-Code alone, with a 16-dimensional edge feature vector carrying 7 QoS dimensions and multi-task heads for component and relationship criticality (Section 4). Our central empirical claim concerns cross-architecture transfer, and we state it at *matched* QoS encoding rather than across the widest available pair of variants: under inductive distribution shift on a shared native multigraph substrate, typed learning leads untyped learning by $\Delta\rho = +0.353$ without QoS edge features (0.439 vs. 0.086) and by $+0.246$ with them (0.608 vs. 0.363), winning 8 of 8 folds in both cases ($p = 0.0078$; Section 7.2). In-distribution the two families are indistinguishable ($\rho = 0.630$–0.660 vs. 0.653–0.725), so typing is an inductive generalization property rather than a fitting advantage. Simultaneously, we establish the honest empirical boundary: against an unparameterized QoS-weighted centrality baseline (`Topo-QoS`), out-of-distribution ranking is matched rather than surpassed ($\Delta\rho = +0.037$, $p = 0.64$), and the critical-set margin ($F_1@K = 0.414$ vs. 0.380) is not statistically significant either, won in only 4 of 8 folds ($p = 0.47$; Section 7.1).
2. **A Formal Typed Architecture Model:** A multigraph representation that derives logical dependencies from physical pub-sub linkages and distinguishes sequential cascade propagation from simultaneous multi-consumer library failures, supplying the typed substrate the predictor consumes (Section 3).
3. **A Standards-Grounded Explanation Layer:** An interpretable Reliability–Maintainability model grounded in ISO/IEC 25010/25019 that turns the predictor's ranked output into an actionable diagnosis, separating single-point-of-failure exposure from error-propagation depth—two distinct failure modes requiring different repairs (Section 5).
4. **Empirical Benchmark, Real-World Evaluation, and Cost Characterization:** An evaluation across seven synthetic topologies (1,545 components) and three open-source reference systems (225 components) under strict graph-view separation, establishing both where typed graph learning helps and the boundary where it does not, with a per-stage cost profile locating the pipeline's cost in its deterministic graph-analysis stage rather than in the learned model (Sections 6–7).

Relationship to the authors' prior work. An earlier, shorter version was presented at a peer-reviewed conference [24], covering the preliminary typed multigraph formulation and the deterministic quality-attribution model on the synthetic corpus alone. This manuscript is a substantially extended version meeting the JSS extension policy. New here are: the entire predictive pathway (the HGT, its 16-D QoS edge encoding, the multi-task masked-loss heads, and every learned result in Section 7); the inductive LOSO protocol and cross-architecture

transfer analysis (Section 7.2); zero-shot evaluation on three open-source reference systems (Section 7.4); the cost characterization answering RQ5 (Section 7.5); the four-oracle convergent-validity taxonomy and graph-view separation (Sections 4.3–4.4); global sensitivity analysis over all ten weight constants (Section 7.3); and the homogeneous-versus-heterogeneous comparison under substrate parity (Section 7.2). Material retained from the conference version is limited to preliminary formalisms in Sections 3 and 5, both restructured and expanded. No companion manuscript from this work is under consideration elsewhere.

1.6. Paper Organization

The remainder of this paper is organized as follows: Section 2 reviews related work on distributed systems dependability, performance engineering, static system analysis, and graph representation learning. Section 3 formalizes the Software-as-a-Graph architectural model, the dependency projection rules, and the typed node features consumed by both pathways. Section 4 presents the Heterogeneous Graph Transformer, its multi-task heads, the simulation oracles that supply its labels, and the input–label independence guarantee. Section 5 introduces the interpretable RM explanation layer. Section 6 describes the experimental setup, benchmark corpus, and evaluation protocols. Section 7 presents empirical results for RQ1–RQ5. Section 8 discusses architectural implications, performance and computational sustainability, threats to validity, limitations, and concluding remarks.

2. Related Work

This work builds upon and connects four foundational research areas: (1) dependability, performance, and sustainability in distributed software systems; (2) static code and system analysis; (3) software quality measurement and multi-criteria evaluation; and (4) graph representation learning and explainable AI (XAI).

2.1. Dependability, Performance, and Sustainability in Distributed Software Systems

The publish–subscribe (pub-sub) and asynchronous event-driven paradigms decouple communicating entities in space, time, and synchronization, enabling elastic scalability and high throughput [7]. Modern middleware standards—such as ROS 2 [1], Apache Kafka [2], DDS [3], and MQTT [4]—govern these exchanges through fine-grained Quality-of-Service (QoS) policies that regulate message durability, transport reliability, priorities, and delivery deadlines. In cloud-native microservice meshes and distributed AI/LLM serving backbones, asynchronous message passing and queueing topologies form the primary communication substrate, directly shaping tail latencies, throughput bottlenecks, and hardware resource utilization.

Prior dependability and performance research has focused predominantly on **runtime mechanisms**, including dynamic consensus protocols, broker clustering, adaptive backpressure throttling, autoscaling, and automated failover. In parallel, **chaos engineering and runtime verification** [19] inject faults or latency into staging or production clusters to observe degradation and recovery. This delivers operational validation that no static method can match, but it requires a fully provisioned cluster, carries the risk of real service disruption, and consumes substantial cluster time per sweep — so it cannot be applied during architectural design or early CI/CD.

Our work addresses the complementary **pre-deployment phase**: predicting systemic cascading vulnerabilities directly from Architecture-as-Code descriptors before runtime infrastructure is provisioned, enabling architectural gating within CI/CD pipelines without a running cluster.

Architecture-Based Reliability Prediction. Predicting dependability from an architectural description before deployment is not a new ambition, and SaG should be read against the tradition that pursued it analytically. Cheung's absorbing-Markov-chain model [25] derives system reliability from component reliabilities and a transfer-of-control graph; Goseva-Popstojanova and Trivedi [26] systematize the state-based, path-based and additive families that followed, and Immonen and Niemelä [27] survey the resulting methods from the architectural perspective. Model-driven descendants such as the Palladio Component Model [28] and layered queueing networks [29] predict performance and reliability from parameterized component models with well-understood solution techniques.

These methods are complementary to ours rather than superseded by it, and the distinction is one of required inputs rather than of accuracy. They need per-component failure probabilities, transition probabilities or service demands — parameters that are themselves estimated from operational profiles, measurement or expert judgement, and that are unavailable in the setting we target (a manifest at commit time, with no telemetry). SaG asks a narrower question in exchange: not what the system's reliability *is*, but which components' failures would propagate furthest through the declared topology. Where those parameters can be obtained, an analytical model answers a stronger question than a ranking does, and we make no claim to displace it.

Data-Driven Failure Prediction and Root-Cause Analysis in Microservices. A large recent literature localizes faults in microservice systems from operational data. Seer [30] and Sage [31] predict and debug QoS violations from traces and hardware telemetry; MicroRCA [32] and TraceRCA [33] localize root causes over service-dependency and trace graphs; DeepTraLog [34] and Eadro [35] combine traces, logs and metrics under graph-based deep models. This line is the closest methodological neighbour to our predictive pathway, and it consistently outperforms what a purely static analysis can achieve — because it observes the running system. That is precisely the boundary: every one of these approaches requires a deployed system emitting traces, logs or metrics, and therefore cannot answer a question posed at design or pull-request time. SaG occupies the pre-deployment complement, and accepts a correspondingly weaker evidential basis: simulated rather than observed failures, and topology rather than behaviour.

2.2. Static Code Analysis (SCA) vs. Static System Analysis (SSA)

Traditional **Static Code Analysis (SCA)** tools (e.g., SonarQube [15]) inspect source code Abstract Syntax Trees (ASTs) within individual services. They evaluate cyclomatic complexity [16], class cohesion, module coupling (e.g., Lack of Cohesion in Methods [LCOM], Coupling Between Objects [CBO]) [17, 18], and code duplication to flag internal code smells and defect-prone modules [36, 37, 38, 39]. However, SCA cannot observe runtime communication topology: it is blind to inter-service messaging channels, message broker queue saturation, and cross-host failure propagation.

To bridge this "Architecture–Code Gap," **Static System Analysis (SSA)** extends static analysis from single-service source code to the global system architecture. By modeling distributed applications, message topics, brokers, execution nodes, and shared libraries as a connected multigraph, SSA propagates code-level quality metrics across architectural dependencies. This allows engineering teams to detect structural anti-patterns [40, 41] and architectural technical debt [42] early during continuous integration (CI/CD) [43, 44], before defective topologies enter production.

2.3. Software Quality Models and Multi-Criteria Evaluation

Software product quality is standardized by the **ISO/IEC 25010:2023** product quality model [9] and the **ISO/IEC 25019:2023** Quality-in-Use model [10]. ISO/IEC 25010:2023 defines three closely intertwined characteristics critical to modern distributed systems:

- **Reliability:** The degree to which a system performs specified functions under stated conditions, comprising Faultlessness, Availability, Fault Tolerance and Recoverability.
- **Maintainability:** The degree of effectiveness and efficiency with which software can be modified, comprising Modularity, Reusability, Analysability, Modifiability and Testability.
- **Performance Efficiency:** Performance relative to resource consumption under stated conditions, comprising Time Behavior (latency, response time), Resource Utilization (CPU, memory, bandwidth), and Capacity.

SaG operationalizes a strict subset of these: Availability and Fault Tolerance under Reliability, and Modularity, Modifiability and Analysability under Maintainability (Section 5.1). Faultlessness, Recoverability, Reusability and Testability are not derivable from deployment topology alone and are outside the scope of this work.

Software engineering measurement explicitly distinguishes between *internal quality* (measured on static artifacts at rest) and *external quality* (measured on executing software systems) [45, 46]. In distributed architectures, architectural debt (such as over-centralized message topics or unreplicated brokers) degrades internal quality and precipitates severe external performance bottlenecks, queue congestion, and outages.

Aggregating multi-attribute structural metrics into an auditable quality score constitutes a classic Multi-Criteria Decision Making (MCDM) problem. The **Analytic Hierarchy Process (AHP)** [47] delivers a structured pairwise-comparison method with an explicit Consistency Ratio ($CR \leq 0.10$) to ensure mathematical soundness in weighting models. This study applies AHP to construct an audited, explainable Reliability–Maintainability (RM) quality baseline, in conjunction with learned graph models.

2.4. Graph Representation Learning and Explainable AI

Network science provides established centrality metrics to identify critical nodes, such as degree, closeness, betweenness centrality [20, 22], articulation points, and PageRank [21, 23]. Foundational studies on network robustness [48], cascading overloads [49], and interdependent networks [50] model how disruptions propagate across connected systems. However, standard network metrics suffer from two major limitations when applied to software architectures:

1. **Dimensional Collapse:** A single centrality scalar cannot distinguish *why* a component is critical—for instance, whether it is a single point of failure (SPOF), an error-propagating cascade hub, or an over-shared library.
2. **Semantic Collapse:** Standard metrics treat all nodes and edges identically. They conflate fundamentally different architectural entities, such as an asynchronous message topic, a shared library, and a physical execution host.

To overcome the limits of hand-engineered metrics, recent research has applied machine learning to network vulnerability (e.g., FINDER [51], DrBC [52], and PowerGraph [53]). However, most available models rely on **homogeneous message passing** (GCN [54], GraphSAGE [55], GAT [56]), which averages signals across all connections indiscriminately. Because distributed software architectures are inherently **heterogeneous** (comprising distinct entity types and relationship rules), homogeneous models blur critical architectural boundaries and fail to generalize out-of-distribution.

Heterogeneous Graph Neural Networks (RGCN [57], HAN [58], HGT [59], MAGNN [60]) resolve this by employing relation-specific transformations. We build upon the **Heterogeneous Graph Transformer (HGT)** architecture [59] to preserve typed relational semantics when forecasting cascading failure blast radii and performance degradation.

Explainable AI (XAI) vs. The Black-Box Barrier. A critical hurdle in applying modern AI to software engineering is the **black-box barrier**: deep neural models output risk scores or continuous embeddings without explaining underlying structural causality. In production software engineering, uninterpretable risk rankings hinder actionable decision-making: developers and SREs cannot determine whether to replicate a host, configure circuit breakers, or refactor shared libraries.

Existing GNN explanation techniques, such as GNNExplainer [61] and PGExplainer [62], identify influential subgraphs through edge masking or parameterized learning. Although useful, these methods explain the model using internal latent representations rather than standardized software engineering concepts. SaG resolves this limitation through a decoupled dual-pathway design: the predictive HGT pathway reveals typed mutual-attention distributions indicating *which* architectural relations propagated the cascade (Section 7.3), while the deterministic explanation layer attributes fragility to standardized ISO/IEC quality sub-characteristics (Section 5), translating raw predictions into actionable, cost-effective remediations.

3. The Software-as-a-Graph (SaG) Architectural Model

This section formalizes the Software-as-a-Graph multigraph representation (Section 3.1), the QoS-aware weighting and logical dependency derivation rules (Section 3.2), the dual graph views (Section 3.3), and the typed node feature encodings (Section 3.4).

3.1. Formal Multigraph Definition

A complex distributed software system is formally modeled as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \tag{1}$$

where:

- V is the set of system entities, partitioned into five disjoint entity types:

$$V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{node}} \cup V_{\text{lib}} \tag{2}$$

- E is the set of directed edges connecting entities.
- $\tau_V : V \to \mathcal{T}_V$ and $\tau_E : E \to \mathcal{T}_E$ are typing functions assigning node and edge categories.
- $w_V : V \to [0, 1]$ and $w_E : E \to [0, 1]$ are weighting functions representing entity criticality and connection strength.

Table 1 summarizes the five entity types and six structural edge types formalized in the SaG model, along with their semantics and representative concrete distributed systems implementations.

Table 1: Entity and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	**Architectural Role**	**Concrete System Examples**
Application (V_{app})	Autonomous process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	`/sensor/lidar`, `orders.payment.completed`
Node (V_{node})	Physical host or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	`librdkafka`, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	**Direction**	**Semantic Meaning**
`PUBLISHES_TO`	App/Library → Topic	Component publishes messages to topic
`SUBSCRIBES_TO`	App/Library → Topic	Component consumes messages from topic
`ROUTES`	Broker → Topic	Broker manages and routes topic traffic
`RUNS_ON`	App/Broker → Node	Process is hosted on physical/virtual host
`CONNECTS_TO`	Node → Node	Physical network link between hosts
`USES`	App → Library	Application links to shared library dependency

Application and Library entities additionally incorporate static code metrics computed via Static Code Analysis (SCA) tools (`cm_*` attributes: lines of code, cyclomatic complexity, coupling between objects, LCOM), linking code-level fragility directly to topological analysis.

3.2. QoS-Aware Weights and Logical Dependency Derivation

In distributed middleware, communication links vary in coupling strength based on their Quality-of-Service (QoS) contracts. For instance, a `RELIABLE` topic with `TRANSIENT_LOCAL` durability binds communicating services substantially more tightly than a `BEST_EFFORT` telemetry stream.

Each topic t carries an intrinsic criticality weight $w(t) \in [0, 1]$ combining its declared QoS semantics with two runtime-stress modulators: payload size and publication frequency:

$$w(t) = \beta \cdot \text{QoS}(t) + \alpha \cdot \text{SizeNorm}(t) + \psi \cdot \text{FreqNorm}(t), \quad (\beta, \alpha, \psi) = (0.75,\ 0.15,\ 0.10) \tag{3}$$

where the QoS term is an AHP-weighted aggregate of the declared contract:

$$\text{QoS}(t) = w_{\text{rel}} \cdot q_{\text{rel}} + w_{\text{dur}} \cdot q_{\text{dur}} + w_{\text{prio}} \cdot q_{\text{prio}}, \quad (w_{\text{rel}}, w_{\text{dur}}, w_{\text{prio}}) = (0.24,\ 0.62,\ 0.14) \tag{4}$$

Here, $q_{\text{rel}}, q_{\text{dur}}, q_{\text{prio}} \in [0, 1]$ represent normalized reliability, durability, and transport-priority scores. Durability dominates because it governs whether data persists across restarts and network partitions. Reliability and

transport priority both govern in-flight delivery quality, with reliability receiving higher weight because unconditional delivery guarantees precede message scheduling. The sub-weight vector is the geometric-mean priority vector of an independently stated Saaty pairwise-comparison matrix with a small, non-zero consistency ratio ($CR \approx 0.016 \leq 0.10$). The modulators are logarithmically compressed: $\text{SizeNorm}(t) = \log_2(1 + \text{bytes})/20$ (a 1 MiB design envelope, representing the practical DDS sample ceiling before RTPS fragmentation dominates) and $\text{FreqNorm}(t) = \log_{10}(1 + \text{Hz})/3$. The weight $w(t)$ is clamped to $[0.01, 1]$ ensuring that best-effort edges remain visible to graph traversals. Every structural communication edge incident on t (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`) inherits $w_E(e) = w(t)$ along with the topic's QoS vector.

The outer split (β, α, ψ) is a declared convex combination whose sensitivity is evaluated in Section 7.3.2.

3.2.1. Logical Dependency Projection (`DEPENDS_ON`)

Structural edges capture explicit deployment connections but omit implicit runtime dependencies. For example, a subscriber depends upon a publisher, yet no direct edge connects them in pub-sub architectures. We therefore derive a single unified semantic relation, `DEPENDS_ON`, directed from *dependent* to *dependency* ("if target fails, source is impacted"), according to the six projection rules detailed in Table 2:

Table 2: The six `DEPENDS_ON` logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent → Dependency)	Derived Weight (w)
1	`app_to_app`	Subscriber → Publisher (via shared Topic, incl. transitive `USES`)	$1 - \prod_{t \in T}(1 - w(t))$
2	`app_to_broker`	Publisher/Subscriber → Broker routing its topics	$1 - \prod_{t \in T}(1 - w(t))$
3	`node_to_node`	Host → Host (lifted from inter-host app dependencies)	Lifted $\max w$
4	`node_to_broker`	Host → Broker (lifted from hosted app dependencies)	Lifted $\max w$
5	`app_to_lib`	Application → Shared Library it `USES`	$H(w_V(\text{app}), w_V(\text{lib}))$
6	`broker_to_broker`	Broker ↔ Broker (shared physical fault-domain colocation, symmetric)	$w_V(\text{node})$

Rules 1 and 2 aggregate the set of topics T connecting a component pair using a probabilistic union rather than a maximum [63, 64, 65]. This guarantees that additional parallel failure vectors increase coupling monotonically while keeping $w \in (0, 1]$. Rule 5 applies the harmonic mean $H(x, y) = 2xy/(x + y)$ [66] to combine the consuming Application's and the shared Library's vertex weights, balancing caller and dependency criticality. Rules 3 and 4 assign the maximum weight among component-level dependencies crossing the host boundary.

3.2.2. Sequential Cascades vs. Simultaneous Blasts

A foundational principle of the SaG model is distinguishing between two fundamentally different degradation modes:

- **Sequential Cascade (Rule 1):** When an application publisher fails, downstream subscribers suffer message starvation. The failure propagates hop by hop through message queues and topic buffers.
- **Simultaneous Blast (Rule 5):** When a shared software library or execution node crashes, all consuming applications and colocated brokers fail *instantaneously* in a single shared-fate event.

Preserving architectural entity types and relation-specific projection rules enables SaG to model both mechanisms, whereas untyped homogeneous graphs collapse them into indistinguishable edges.

Rule 6 is intentionally the only symmetric projection rule. It does not imply that one broker functionally depends on another, but rather that two brokers colocated on the same host share that host's physical failure domain. This follows the same simultaneous-blast principle as Rule 5, which is why the derived weight equals the shared Node's weight and the relation is bidirectional. In production middleware deployments, colocated brokers compete for host resources (CPU cores, page cache, file descriptors, and NIC bandwidth); a host outage takes down all colocated instances simultaneously. Operational best practices for Kafka, RabbitMQ, and EMQX recommend distributing brokers across fault domains. Rule 6 does not model directional intra-cluster broker coupling (e.g., partition replication, controller quorum election, federation, or shovel links), which do not require physical colocation; extending the schema to capture these interactions

is reserved for future work. In our evaluation corpus, Rule 6 applies in four of the eight cached scenarios (the seven synthetic topologies of Table 4 plus the ATM case study of Section 6.1) and contributes only 12 directed edges in total. Because the simulation oracles operate strictly on $G_{\text{structural}}$ (Section 4.4), Rule 6 has zero influence on ground-truth failure labels.

3.3. Dual Graph Views and Architectural Layers

The SaG framework maintains two distinct representations of the system:

1. **Structural Graph ($G_{\text{structural}}$):** The raw deployment graph containing physical and structural relations (such as `PUBLISHES_TO`, `ROUTES`, `RUNS_ON`, and `USES`). Discrete-event simulators consume this view exclusively to execute unbiased failure injections (Section 4.3).
2. **Analysis Graph (G_{analysis}):** The projected graph containing derived `DEPENDS_ON` edges annotated with QoS weights and ingested SCA code metrics. All GNN feature representations, graph embeddings, and analytical metrics are computed on G_{analysis}.

Figure 2 illustrates this duality on a running example, contrasting the raw structural graph against the derived `DEPENDS_ON` projection.

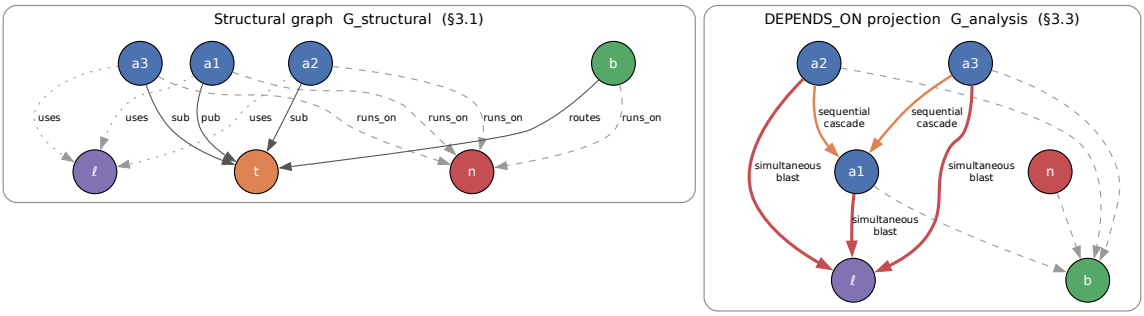

Figure 2: Running example: the raw structural graph (left) and the `DEPENDS_ON` projection derived from it (right). The projection makes implicit runtime dependencies explicit—a subscriber depends on the publishers of its topics even though no structural edge joins them—while the simulators continue to operate on the structural view alone.

G_{analysis} is further structured into four analytical layers (Application, Middleware, Infrastructure, and Global System), enabling evaluation of criticality at subsystem levels, consistent with hierarchical frameworks such as MIL-STD-498 [67].

3.4. Typed Node Feature Encoding

Both pathways read the same typed node properties from G_{analysis}: the predictive pathway (Section 4) projects them per entity type before heterogeneous message passing, and the explanation layer (Section 5) aggregates them into its quality profile. All five entity types share indices 0–17, a common block of topological metrics (in/out degree, betweenness, closeness, reverse PageRank, clustering coefficient, articulation score, bridge load) produced by the deterministic analysis stage whose cost is characterized in Section 7.5. Type-specific features extend that block:

- **Application (23 dims):** indices 18–22 add source-code metrics from SCA — lines of code, cyclomatic complexity, Martin's instability $I_{\text{code}} = C_e/(C_a + C_e)$ [68], Lack of Cohesion in Methods, and the composite Code Quality Penalty (CQP).
- **Library (25 dims):** the Application block plus two library-specific blast-radius drivers (23–24): the normalized size of the transitive reverse-`USES` closure, and the normalized count of distinct subscribers reachable from topics published within that closure.

- **Broker (19 dims):** index 18 is normalized queue buffer capacity.
- **Topic (22 dims):** indices 18–21 are publisher count, subscriber count, log message frequency $\log(1 + \text{freq})$, and ordinal QoS criticality.
- **Infrastructure Node (20 dims):** indices 18–19 are normalized CPU core allocation and physical memory.

4. Graph Learning for Failure-Impact Prediction

Cascading failure impact in distributed software systems is inherently non-linear, multi-hop, and relation-dependent. Outages propagate not merely based on neighbor count, but through architectural relations and dependencies extending multiple hops beyond the initial fault. Whether a closed-form combination of standard centrality metrics can capture these compound dynamics is an empirical question rather than a settled one. The primary predictive pathway of Section 1.2 therefore employs a learned graph model, and Section 7.1 evaluates it against exactly such a closed-form baseline — which, on out-of-distribution ranking, it does not significantly surpass.

This section details the Heterogeneous Graph Transformer (HGT) architecture and its typed edge encodings (Section 4.1), the multi-task prediction heads and dimension-masked loss formulation (Section 4.2), the ground-truth simulation oracles (Section 4.3), and the input–label independence guarantee that prevents data leakage (Section 4.4).

4.1. Heterogeneous Graph Transformer Architecture

Because distributed systems comprise heterogeneous entity types (Applications, Libraries, Brokers, Topics, Infrastructure Nodes) and diverse interaction semantics (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, `USES`, `DEPENDS_ON`), we employ a three-layer **Heterogeneous Graph Transformer (HGT)** architecture [59], implemented within PyTorch Geometric [69], with hidden dimension $D = 64$ and $H = 4$ attention heads. This architecture ensures that typed relations, rather than simple adjacency, govern failure-impact forecasting.

4.1.1. Continuous-Categorical Edge Feature Encoding (16-D)

To capture continuous QoS constraints and channel semantics, SaG encodes each directed edge $e = (u, v)$ as a 16-dimensional continuous-categorical vector $e_{uv} \in \mathbb{R}^{16}$. Index 0 is the scalar coupling weight $w_E(e) \in (0, 1]$ of Section 3.2; index 1 is the normalized count of simple paths through e in G_{analysis}; indices 2–8 one-hot encode the seven structural and derived relations (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, `USES`, `DEPENDS_ON`). Indices 9–15 carry the seven explicit QoS dimensions, active on `PUBLISHES_TO` and `SUBSCRIBES_TO` edges and zeroed elsewhere: reliability (0 best-effort, 1 reliable); durability (0 volatile, 0.5 transient-local, 0.6 transient, 1 persistent); message priority (0, 0.33, 0.66, 1); a deadline-active flag; $\log_{10}(1 + \text{deadline\_ns}/10^6)$; $\log_{10}(1 + \text{max\_blocking\_ms})$; and a heterogeneity flag set when the edge's QoS triple departs from its scenario's modal profile.

An edge projection module maps e_{uv} into the hidden space: $e'_{uv} = W_{\text{edge}} e_{uv}$. Prior to relational attention computation, this projection vector is incorporated directly into the target node representation: $\tilde{h}_v = h_v + e'_{uv}$.

4.1.2. Type-Specific Projection and Heterogeneous Message Passing

For each source node u and target node v connected by meta-relation $\tau(e) = (\tau(u), \phi(e), \tau(v))$:

1. **Type-Specific Projection:** Node feature vectors x_v (of dimension 19–25 depending on entity type $\tau(v)$) are mapped into the shared D-dimensional hidden space:

$$h_v^{(0)} = \text{LayerNorm}\big(\text{GELU}(W_{\tau(v)} x_v)\big) \tag{5}$$

2. **Relational Mutual Attention:** Type-parameterized Query (Q), Key (K), and Value (V) projections calculate relation-specific attention across H heads:

$$\text{Attn}(u, e, v) = \underset{\forall u \in \mathcal{N}(v)}{\text{Softmax}} \left(\frac{K(u) W_{\text{att},\phi(e)} Q(\tilde{h}_v)^\top}{\sqrt{D/H}} \right) \tag{6}$$

$$\text{Msg}(u, e, v) = V(u) W_{\text{msg},\phi(e)} \tag{7}$$

3. **Bidirectional Message Passing:** To capture downstream consumer starvation and upstream backpressure simultaneously, message passing is executed over both forward and transposed relation views (G_{analysis} and $G_{\text{analysis}}^\top$).
4. **Residual Aggregation and Layer Normalization:** Target node representations are updated across layers $l \in \{1, \ldots, L\}$ via residual connections and layer normalization:

$$h_v^{(l)} = \text{LayerNorm} \left(h_v^{(l-1)} + \text{Dropout} \left(\sum_{u \in \mathcal{N}(v)} \text{Attn}(u, e, v) \cdot \text{Msg}(u, e, v) \right) \right) \tag{8}$$

Training Protocol and Optimization Hyperparameters. Models are optimized end-to-end using AdamW with initial learning rate $\eta = 3 \times 10^{-4}$, weight decay 10^{-4}, and dropout probability $p = 0.10$ applied post-attention. Learning rates follow a cosine annealing schedule with warm restarts (CosineAnnealingWarmRestarts, $T_0 = 75$, $T_{\text{mult}} = 2$, $\eta_{\min} = 3 \times 10^{-6}$). Training executes for a maximum of 300 epochs with early stopping governed by a patience of 30 epochs monitored on validation loss over labeled nodes. Inductive subgraphs are processed per scenario using full-graph inductive packing without mini-batch subsampling, with validation masks isolating held-out nodes to prevent information leakage across data splits. Five independent random seeds $\{42, 123, 456, 789, 2024\}$ are evaluated across all runs, redrawing both partition masks and initializations. *Selection protocol:* the architectural hyperparameters ($D = 64$, $H = 4$, dropout, learning rate and schedule) and the loss coefficients of Equation (9) were fixed a priori from values conventional for HGT [59] and were not tuned against any evaluation reported in this paper; no search over them was performed, on either the in-distribution test split or the LOSO folds. This avoids selection leakage, at the cost of leaving open whether either family is reported near its own optimum — a comparison between untuned configurations, which we state rather than treat as a like-for-like optimum comparison.

4.2. Multi-Task Prediction Heads and Dimension Masking

From the final node embeddings $h_v^{(L)}$, SaG utilizes specialized multi-task prediction heads:

- **Reliability Head:** $\hat{R}(v) = \sigma(\text{MLP}_R(h_v)) \in [0, 1]$
- **Maintainability Head:** $\hat{M}(v) = \sigma(\text{MLP}_M(h_v)) \in [0, 1]$
- **Composite Failure Impact Head:** $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{R}(v) \parallel \hat{M}(v))) \in [0, 1]$
- **Relationship Criticality Head:** $\hat{Q}(u, v) = \sigma(\text{TypedEdgeEncoder}_{\phi(e)}(h_u, h_v, e_{uv})) \in [0, 1]$

4.2.1. Dimension-Masked Loss Formulation

The combined optimization objective integrates regression accuracy, multi-task dimension learning, ranking fidelity, pairwise ordering, and edge prediction:

$$\mathcal{L} = \mathcal{L}_{\text{composite}} + 0.5 \cdot \mathcal{L}_{\text{dimension}} + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}} + 0.3 \cdot \mathcal{L}_{\text{edge}} + \lambda_{\text{RM}} \cdot \mathcal{L}_{\text{consistency}} \tag{9}$$

where $I^*(v)$ is the simulated cascade impact defined by the primary oracle (Section 4.3), $\mathcal{L}_{\text{composite}} = \text{MSE}(\hat{I}^*(v), I^*(v))$, $\mathcal{L}_{\text{rank}}$ is the ListMLE ranking loss [70], $\mathcal{L}_{\text{pairwise}}$ is margin-ranking loss, and $\mathcal{L}_{\text{consistency}} = \text{MSE}\big([\hat{R}(v), \hat{M}(v)]_{v \in \text{unlabeled}}, [R_{\text{RM}}(v), M_{\text{RM}}(v)]_{v \in \text{unlabeled}}\big)$ regresses predicted heads toward the diagnostic

pathway's baseline (Section 5) on unlabeled nodes. Headline results use $\lambda_{\text{RM}} = 0$, guaranteeing that the predictive and explanatory pathways remain strictly independent.

Dimension Masking: Because dynamic cascade simulation ($I^*(v)$ via `FaultInjector`) observes runtime failure reachability rather than source-code maintainability, maintainability ground truth is unobserved during dynamic simulation. A separate change-propagation oracle $I_M(v)$ evaluates static structural change ripple at the Validate stage, but is never used as a training label to avoid circular supervision. We introduce a boolean dimension mask $m = [m_R, m_M] = [1, 0]$:

$$\mathcal{L}_{\text{dimension}} = \frac{1}{\sum_d m_d} \sum_{d \in \{R,M\}} m_d \cdot \text{MSE}(\hat{d}(v), d^*(v)) \tag{10}$$

This mask ensures the unobserved maintainability head is not artificially penalized or driven toward zero during backpropagation.

4.2.2. Domain-Reweighted Criticality (Q_{domain})

To ground predictions in ISO/IEC 25019 Context of Use ($\vec{\omega} = [q_R, q_M]^\top$), the composite score can be evaluated as:

$$Q_{\text{domain}}(v) = q_R \cdot \hat{R}(v) + q_M \cdot M_{\text{static}}(v) \tag{11}$$

where $M_{\text{static}}(v)$ is obtained directly from the structural analyzer's maintainability baseline, combining learned dynamic reliability with static source-code maintainability. Because maintainability is unobserved in dynamic simulation ($m = [1, 0]$), headline results report $\hat{I}^*(v)$ directly, while domain reweighting sensitivity is evaluated against the static RM baseline in Section 7.3.

4.3. Ground-Truth Simulation Oracles

To evaluate predictive accuracy prior to deployment without relying on production runtime telemetry, SaG executes discrete-event failure simulations over the raw structural multigraph $G_{\text{structural}}$. We establish a formal taxonomy of four component-level oracles and one relationship-level oracle:

- **Cascade Reachability Oracle ($I^*(v)$):** Implemented via `FaultInjector`, it crashes component v, propagates outages across dependent topics, brokers and links by breadth-first traversal, and returns the fraction of surviving subscriber feeds severed. Publisher loss is weighted by publication rate and feed losses scaled by a QoS ladder ($\times 1.2$ `RELIABLE`, $\times 1.15$ high/urgent priority, $\times 1.05$ medium) before clamping to $[0, 1]$. This is the **primary continuous target label** throughout.

- **Multi-Metric Composite Oracle ($I_{\text{comp}}(v)$):** Implemented via `FailureSimulator`, this oracle evaluates a multi-faceted failure impact vector:

$$I_{\text{comp}}(v) = 0.35 \cdot \Delta\text{Reachability} + 0.25 \cdot \Delta\text{Fragmentation} + 0.25 \cdot \Delta\text{Throughput} + 0.15 \cdot \Delta\text{FlowDisruption} \tag{12}$$

  where each term is weighted by operational severity $s(t) = w(t) \cdot \text{rate}(t)$. $I_{\text{comp}}(v)$ serves as the canonical oracle for architectural quality gate verification.

- **Dynamic Queue-Flow Oracle ($I_{\text{dyn}}(v)$):** Implemented via `MessageFlowSimulator` using the SimPy framework [71], this oracle simulates message emission rates, stochastic network latencies, broker buffer saturation, and queue drops under fault injection. It extracts the drop in delivered message rate suffered by surviving consumers.

- **Change-Propagation Oracle ($I_M(v)$):** Implemented via `ChangePropagationSimulator`, this oracle executes a deterministic breadth-first traversal of the reversed dependency graph to quantify maintenance change impact:

$$I_M(v) = 0.45 \cdot \text{ChangeReach}(v) + 0.35 \cdot \text{WeightedChangeImpact}(v) + 0.20 \cdot \text{NormalizedChangeDepth}(v) \tag{13}$$

- **Relationship (Edge) Removal Oracle ($I_{\mathbf{edge}}(u,v)$):** Evaluates the systemic impact of severing an individual dependency or communication channel while keeping endpoint components operational. Writing $\bar{I}_{\text{comp}}(G) = |V|^{-1} \sum_{v \in V} I_{\text{comp}}(v; G)$ for the mean composite impact over a graph G:

$$I_{\text{edge}}(u,v) = \bar{I}_{\text{comp}}\big(G \setminus \{(u,v)\}\big) - \bar{I}_{\text{comp}}(G) \tag{14}$$

Withholding declared topic criticality from labels. `FailureSimulator` can blend a declared `Topic.criticality` into its severity term; this is disabled, because that field is a GNN input feature (`topic_qos_criticality_ord`) and consuming it would score the predictor against a transformation of its own input.

Primary Oracle Declaration and Role Assignment. Because the three reliability-facing oracles (I^*, I_{comp}, I_{dyn}) measure distinct operational constructs, we designate $I^*(v)$ **(`FaultInjector`) as the primary oracle** for all predictive ranking results (Tables 6–8, RQ1–RQ3). $I_{\text{comp}}(v)$ is reserved for Validate-stage quality gates, $I_{\text{dyn}}(v)$ serves as an independent convergent-validity probe, and $I_M(v)$ serves as a structural maintainability reference.

Cross-Oracle Convergent Validity and Critical-Set Bounds. Measured across seven benchmark scenarios and five random seeds on the Application population, the mean Spearman rank correlation is $\rho = 0.883$ for (I_{dyn}, I^*), $\rho = 0.468$ for (I_{comp}, I^*), and $\rho = 0.465$ for $(I_{\text{comp}}, I_{\text{dyn}})$. The strong rank agreement ($\rho = 0.883$) between the behavioral queue-flow oracle and the topological cascade injector provides independent convergent evidence across distinct simulation paradigms. However, agreement on the top-K critical set ($K = 0.2n$) is more conservative (mean Jaccard overlap of 0.42 for the strongest pair vs. 0.111 expected by chance), highlighting the intrinsic sensitivity of discrete thresholding in non-linear cascades. Consequently, results established against one oracle are never transferred to another; every evaluation metric explicitly references its underlying simulation oracle.

4.4. Input–Label Independence Guarantee

To eliminate data leakage and ensure rigorous evaluation, SaG enforces strict architectural separation between inputs and labels:

- **Feature Space:** Constructed exclusively from G_{analysis} using static structural topology, static code analysis (SCA) metrics, and declared QoS contracts.
- **Label Space:** Evaluated exclusively on raw $G_{\text{structural}}$ through independent simulation oracles (`FaultInjector`, `FailureSimulator`, `MessageFlowSimulator`).

No simulation outputs, failure trace histories, or dynamic execution telemetry are ever exposed as input features to the GNN or the explanation layer.

5. The Explanation Layer: Standards-Grounded Criticality Attribution

The predictor of Section 4 answers *where* to act. It does not answer *what to do*, and neither does the oracle that scores it — both return impact, not cause attributed in standardized quality terms. A component may be critical because it is a single point of failure, because it propagates errors widely, or because it is a high-coupling maintenance bottleneck. These diagnoses call for distinct architectural repairs: replicating the host or broker, decoupling the topic, or refactoring the module. This section presents the layer supplying that diagnosis. SaG decomposes component and relationship criticality into a standards-grounded quality profile, computed over the same typed node features (Section 3.4) but sharing no parameters with the predictor, and applied to whatever the predictor flagged — triage rather than data flow (Figure 1).

5.1. Grounding in ISO/IEC Standards

In accordance with **ISO/IEC 25010:2023** (Product Quality Model) [9], **ISO/IEC 25019:2023** (Quality-in-Use) [10], and **ISO/IEC 25022:2016** (Measurement of Quality-in-Use) [72], SaG formalizes two primary criticality constructs:

- **Component Criticality (D_1):** The degree to which the sudden failure, unexpected termination, or severe degradation of an individual component reduces the system's capacity to deliver required services within its operational context of use.
- **Relationship Criticality (D_2):** The degree of systemic service degradation resulting from the severance, partitioning, or failure of a specific dependency or communication channel while both endpoint components remain operational.

Criticality is evaluated across two orthogonal quality characteristics: **Reliability (R)** and **Maintainability (M)**. The two are separated because they imply different repairs, not because they predict different runtime quantities: this layer attributes structural fragility and does not forecast latency, throughput or queue occupancy. Table 3 outlines this Reliability–Maintainability (RM) quality decomposition, mapping ISO/IEC sub-characteristics to architectural questions, underlying graph metrics, and targeted engineering remediations.

Table 3: The Reliability–Maintainability (RM) quality decomposition.

Dimension	Sub-Characteristic	Architectural Question	Underlying Graph Metrics	Role / Remediation
Reliability (R)	**Fault Tolerance** (FT)	How broadly does failure propagate?	Reverse PageRank on $G^\top$, in-degree, cascade depth	Reliability Eng.: add redundancy, circuit breakers
	Availability (A)	Is this a single point of failure?	Directed articulation score (raw + QoS-weighted), bridge ratio, connectivity degradation	DevOps/SRE: replicate host/broker
Maintainability (M)	**Modularity/Modifiability**	How complex and coupled is this?	Betweenness, QoS-weighted out-degree, Code Penalty, coupling-risk imbalance, inverse clustering	Architect: refactor code, decouple

Coverage Scope: SaG focuses specifically on Reliability and Maintainability. Safety (which requires domain-specific hazard logs, such as ISO 26262 Automotive Safety Integrity Level [ASIL] ratings) and Security (which requires explicit threat models, such as STRIDE [Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege]) fall outside purely structural topology analysis and are reserved for domain-specific extensions.

5.2. Composite Quality Score Formulation

All raw topological and code metrics are rank-normalized to $[0, 1]$. Quality sub-characteristics are formulated hierarchically using the Analytic Hierarchy Process (AHP) [47]:

1. **Fault Tolerance ($FT(v)$):** Measures error cascade potential on the transpose graph $G^\top_{\text{analysis}}$ (where edges follow failure propagation from dependency to dependent):

$$FT(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{Deg}_{\text{in}}(v) + 0.25 \cdot \text{CDPot}_{\text{enh}}(v) \tag{15}$$

   where $\text{RPR}(v)$ is Reverse PageRank, $\text{Deg}_{\text{in}}(v)$ is normalized in-degree, and $\text{CDPot}_{\text{enh}}(v)$ is the enhanced Cascade Depth Potential term.
2. **Availability ($A(v)$):** Identifies structural single points of failure (SPOFs) across five terms — directed articulation severity, its QoS-weighted variant, edge-level irrecoverability, connectivity degradation, and the component's own QoS weight:

$$A(v) = 0.2563 \cdot \text{AP}_c^{\text{dir}}(v) + 0.1998 \cdot \text{QSPOF}(v) + 0.1998 \cdot \text{BR}(v) + 0.2563 \cdot \text{CDI}(v) + 0.0878 \cdot w(v) \tag{16}$$

   where $\text{AP}_c^{\text{dir}}(v)$ is Directed Articulation Point severity, $\text{QSPOF}(v)$ is QoS-weighted Single Point of Failure severity, $\text{BR}(v)$ is Bridge Ratio (edge-level irrecoverability), $\text{CDI}(v)$ is Connectivity Degradation Index, and $w(v)$ is the component's intrinsic QoS weight.
3. **Reliability ($R(v)$):** Blends Fault Tolerance and Availability hierarchically:

$$R(v) = r_\alpha \cdot FT(v) + (1 - r_\alpha) \cdot A(v), \quad r_\alpha = 0.36 \tag{17}$$

   The blend weight is written r_α throughout to distinguish it from the payload-size coefficient α of Equation (3); the two are unrelated parameters and are screened separately in Table 11. Intra-dimension pairwise comparison matrices are audited against Saaty's consistency ratio and measure $CR = 0.001$ (Fault Tolerance), $CR = 0.001$ (Availability), and $CR = 0.000$ (Maintainability) — all well within the $CR \leq 0.10$ acceptability threshold. The shipped intra-dimension weights apply a $\lambda = 0.70$ shrinkage blend between the raw AHP-derived vector and a uniform prior (Section 7.3 reports ranking sensitivity to λ).

4. **Maintainability ($M(v)$):** Evaluates structural coupling combined with code-level static analysis across five terms — betweenness, QoS-weighted efferent coupling, the Code Quality Penalty, an afferent/efferent coupling-risk imbalance term, and inverse clustering:

$$M(v) = 0.35 \cdot \mathrm{BT}(v) + 0.30 \cdot w_{\mathrm{out}}(v) + 0.15 \cdot \mathrm{CQP}(v) + 0.12 \cdot \mathrm{CouplingRisk}_{\mathrm{enh}}(v) + 0.08 \cdot (1 - \mathrm{CC}(v)) \quad (18)$$

where $\mathrm{BT}(v)$ is Betweenness Centrality, $w_{\mathrm{out}}(v)$ is QoS-weighted efferent coupling (out-degree), $\mathrm{CQP}(v)$ is the Code Quality Penalty, $\mathrm{CouplingRisk}_{\mathrm{enh}}(v)$ is an afferent/efferent coupling-risk imbalance term, and $\mathrm{CC}(v)$ is the local Clustering Coefficient.

The baseline composite quality score $Q(v)$ combines both dimensions:

$$Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v) \quad (19)$$

When evaluating under a specific ISO/IEC 25019 Context of Use vector $\vec{\omega} = [q_R, q_M]^\top$, the score is reweighted dynamically:

$$Q_{\mathrm{domain}}(v) = q_R \cdot R(v) + q_M \cdot M_{\mathrm{static}}(v) \quad (20)$$

Components are categorized into adaptive criticality tiers using box-plot quartile thresholds:

- **CRITICAL:** $Q > Q_3 + 1.5 \cdot \mathrm{IQR}$
- **HIGH:** $Q_3 < Q \leq Q_3 + 1.5 \cdot \mathrm{IQR}$
- **MEDIUM:** $Q_1 < Q \leq Q_3$
- **MINIMAL:** $Q \leq Q_1$

This yields a directly actionable distinction: a component scoring high on A but low on FT is a single point of failure calling for horizontal replication, whereas one scoring high on FT is an error-cascade hub calling for circuit breakers, rate limiting and bulkhead isolation. Whether acting on these diagnoses improves any measured runtime property is not evaluated here (Section 8.4).

6. Experimental Setup

6.1. Datasets and System Corpus

The evaluation corpus comprises 1,770 components distributed across ten distinct system architectures, as detailed in Table 4:

Table 4: Experimental evaluation corpus.

Dataset / Architecture	System Paradigm	$\lvert V\rvert$	$\lvert V_{\mathbf{app}}\rvert$	Topics	Brokers	Hosts	Libs	$\lvert E\rvert$
Autonomous Vehicle (AV)	ROS 2 Cyber-Physical	152	80	40	4	8	20	797
Enterprise Pub-Sub	Kafka Event Mesh	520	300	120	10	40	50	3,245
Financial Trading	Low-Latency Pub-Sub	124	60	35	5	6	18	580
Healthcare Integration	HL7/FHIR Event Mesh	98	50	25	3	8	12	400
Hub-and-Spoke Enterprise	Broker-Centric Messaging	139	70	30	2	12	25	797
IoT Smart City	MQTT Telemetry Mesh	326	200	80	6	30	10	1,322
Microservices Mesh	Cloud-Native Services	186	90	45	6	15	30	680
Autoware.universe [73]	Real-World ROS 2 Autoware	75	32	24	3	6	10	179
Cloud Microservices [74]	Real-World GCP Boutique	60	22	20	4	6	8	128
Train-Ticket [75]	Real-World Microservices	90	41	30	3	8	8	162
Total		**1,770**						

Here, $|V|$ is the sum of all five entity-type counts per scenario, totaling exactly 1,770 components. $|E|$ counts every raw structural relationship instance recorded in the scenario specification (`PUBLISHES_TO`,

SUBSCRIBES_TO, ROUTES, RUNS_ON, CONNECTS_TO, USES) — the native substrate that simulation oracles traverse — rather than the derived DEPENDS_ON projection constructed for GNN training.

Three real-world architectures were transcribed from authentic open-source repositories using dedicated architectural adapters. The seven synthetic scenarios were produced by a parameterized topology generator: each is fully defined by a committed configuration specifying a random seed, per-entity-type counts, seven-number summaries (mean, median, standard deviation, minimum, maximum, Q_1, Q_3) for application publish and subscribe fan-out, applications per host, library fan-in, topic payload size, and categorical distributions over the three QoS dimensions. Graph degree distributions and clustering emerge directly from these parameters rather than being synthetically forced. Table 5 reports the generative parameters governing each topology's shape; complete configurations are included in the replication package.

Table 5: Generative parameters of the seven synthetic evaluation scenarios. Counts, seed and fan-out figures are read directly from the committed configurations. The modal QoS column gives the most common reliability/durability/priority value and the range of topic shares carrying them, computed from the committed topology rather than the config's declared QoS targets, which domain-driven assignment does not always realize (Section 6.1).

Scenario	Config	Seed	Counts	Pub	Sub	Modal QoS (R/D/P)
Autonomous Vehicle (AV)	scenario_01_autonomous_vehicle	1001	80/40/4/8/20	2.5	5.0	RELIABLE/TRANSIENT_LOCAL/HIGH (85–100%)
Enterprise Pub-Sub	scenario_07_enterprise_xlarge	7007	300/120/10/40/50	3.0	4.5	RELIABLE/TRANSIENT_LOCAL/MEDIUM (100–100%)
Financial Trading	scenario_03_financial_trading	3003	60/35/5/6/18	4.0	6.0	RELIABLE/PERSISTENT/CRITICAL (51–83%)
Healthcare Integration	scenario_04_healthcare	4004	50/25/3/8/12	2.5	3.0	RELIABLE/PERSISTENT/MEDIUM (60–76%)
Hub-and-Spoke	scenario_05_hub_and_spoke	5005	70/30/2/12/25	2.0	7.0	RELIABLE/TRANSIENT_LOCAL/MEDIUM (100–100%)
IoT Smart City	scenario_02_iot_smart_city	2002	200/80/6/30/10	2.0	1.5	BEST_EFFORT/VOLATILE/LOW (56–79%)
Microservices Mesh	scenario_06_microservices	6006	90/45/6/15/30	1.5	2.0	RELIABLE/TRANSIENT_LOCAL/MEDIUM (100–100%)

QoS diversity across the corpus. Per-topic QoS profiles come from domain-keyed lookups that override the generic categorical distributions. Three domains (`hub-and-spoke`, `microservices`, `enterprise`) receive an identical (reliability, durability, priority) triple across all topics and a fourth (`av`) collapses four of five entries to one profile, giving 85%–100% concentration at the modal triple against 51%–79% for the genuinely multi-valued domains (`finance`, `healthcare`, `iot`); see Table 5. The scalar coupling weight $w(t)$ therefore has standard deviation ≈ 0.02 in those four scenarios against 0.19–0.28 in the other three. This bounds what our QoS ablations can establish (Sections 7.3 and 8.3); we report it as a corpus property rather than forcing artificial variation.

Role of the ATM case study. An additional Automated Teller Machine network scenario serves as the eighth LOSO fold and as the substrate for the attention analysis of Section 7.3. It is omitted from Table 4 and from the 1,770-component total because it was authored as an illustrative walkthrough and its specification lacks the seven-number summaries the other scenarios carry — so it participates in the headline LOSO result without being characterized alongside the rest of the corpus, which we note as a reproducibility limitation. Every LOSO evaluation therefore reports eight folds: the seven synthetic topologies of Table 4 plus ATM. The three real-world systems are never LOSO folds; they are withheld entirely and used only for the zero-shot transfer evaluation of Section 7.4.

6.1.1. Reproducibility of the Corpus

The benchmark corpus is designed to be fully regenerable rather than merely statically archived. Each dataset is deterministically generated from its configuration file via:

```
python cli/generate_graph.py batch -input-dir data/scenarios -output-dir <dir>
```

A companion manifest records the random seed, entity counts, git commit hash, and a SHA-256 cryptographic digest for each emitted topology. Continuous integration regression tests assert that every committed dataset regenerates *byte-identically* from its configuration and that all disk digests match the manifest. This guarantees that third parties can reproduce the exact graphs used in our experiments, rather than simply sampling from similar distributions.

6.2. Baselines and Evaluated Predictors

We evaluate four primary predictor configurations, contrasting the proposed learned architectures with training-free topological baselines:

1. **HGL / HGL-QoS** (proposed): Relation-specific Heterogeneous Graph Transformers (Section 4), evaluated both without and with explicit 7-dimensional continuous-categorical QoS edge features on the native multigraph.
2. **GL / GL-QoS** (homogeneous baseline): Homogeneous Graph Attention Networks (GAT) [56] trained on the identical native multigraph substrate with per-type input projections but untyped, single-relation message passing — isolating the specific performance gain conferred by relation typing.
3. **Topo-QoS** (baseline): Training-free, QoS-weighted topological centrality baseline evaluated on the derived application flow projection.
4. **Topo-BL** (baseline): Training-free structural centrality combining unweighted betweenness centrality and articulation point scoring on the flow projection.

In addition, the out-of-distribution evaluation (Table 8) reports **RM / $Q(v)$** (the deterministic hierarchical quality attribution model of Section 5) as a diagnostic reference baseline. RM is not fitted to rank failure impact; its inclusion demonstrates how much learned relational prediction adds over static structural attribution (Section 1.2). Furthermore, deterministic RM scoring drives every sensitivity sweep in Section 7.3, where closed-form formulations isolate parameter effects from neural training stochasticity.

6.2.1. Evaluation Substrates and Parity Guarantee

Predictors in this study operate under strict substrate parity within their respective families:

- **Graph Learning Models (GL, GL-QoS, HGL, HGL-QoS):** All learned neural predictors ingest the complete native typed multigraph across all five entity types in both in-distribution (Table 6) and out-of-distribution Leave-One-Scenario-Out (Table 8) evaluations. Node type reaches homogeneous GL only through its per-type input projection layer, while message passing uses untyped GATConv with shared weights across all edges; in contrast, heterogeneous HGL employs relation-specific HGTConv weight matrices per edge triple alongside edge-type encodings. This guarantees that empirical comparisons between GL and HGL isolate relation-specific parameterization rather than multi-entity topological visibility.

- **Training-Free Topological Baselines (Topo-BL, Topo-QoS):** Topological baselines are evaluated on the derived Application–Library `DEPENDS_ON` projection (Section 3.2). This projected substrate is necessary because in raw publish–subscribe multigraphs, Application nodes never route messages directly, resulting in near-zero betweenness and bridge ratios that yield degenerate, uninformative scores.

- **Ground-Truth Independence Guarantee:** Crucially, **no predictor in either group accesses $G_{\textbf{structural}}$**: that raw topology is strictly reserved as the substrate for ground-truth simulation oracles (Section 4.4), a guarantee formally verified by `tests/test_independence_guarantee.py`.

Regardless of substrate, all variants are scored on an identical, independently resolved Application node set (V_{app}, Section 6.3).

6.3. Evaluation Metrics and Protocols

Figure accessibility. All figures in this paper use the high-contrast, colorblind-safe Okabe–Ito palette together with distinct marker, hatching and node-shape encodings, so that every distinction carried by colour is also carried by form and remains legible in monochrome.

- **Ranking Precision:** Evaluated via Spearman rank correlation (ρ) and Kendall's rank correlation (τ) between predicted component rankings and ground-truth simulated impact $I^*(v)$ from the primary oracle (Section 4.3).

- **Critical-Set Identification:** Measured via $F_1@K$, Precision@K, and Recall@K for top-K critical components, where $K = \text{round}(0.20 \cdot |V_{\text{app}}|)$. Because predicted and ground-truth sets both contain exactly K elements, Precision, Recall, and F_1 coincide identically as the top-K set overlap.

- **Statistical Significance:** Assessed through paired Wilcoxon signed-rank tests [76] ($p < 0.05$) and non-parametric bootstrap 95% confidence intervals over folds ($B = 2{,}000$) [77, 78], reported for every out-of-distribution predictor in Table 8 and for the paired contrasts in Section 7.1. Given the sample size, we regard those intervals rather than the p-values as the primary evidence. The unit of analysis is the scenario (in-distribution, $n = 7$) or the fold (LOSO, $n = 8$), which places the design at the floor of its own resolving power: the smallest attainable two-sided p is 0.0156 at $n = 7$ and 0.0078 at $n = 8$, so only a near-unanimous sign pattern can register at all, and a genuine effect of moderate size is indistinguishable from noise at this sample size. We report p-values uncorrected and treat them as exploratory. Under a family-wise correction no in-distribution comparison in Table 7 can reach $\alpha/6 = 0.0083$ regardless of the data, and the unanimous 8/8 LOSO results survive a six-comparison correction ($\alpha/6 = 0.0083$) but not a seven-comparison one ($\alpha/7 = 0.0071$). Directional consistency across folds, rather than any single p-value, is what carries the out-of-distribution claims.

6.3.1. Evaluation Population

Every predictor within a given evaluation table is scored on an identical node population, resolved strictly from scenario topology and simulation ground truth — never from any model's predictions. Unless otherwise noted, this population is the **Application** set (V_{app}). This aligns with the framework's primary objective (forecasting application-layer cascading failures) and ensures a fair common denominator across both typed and untyped predictors. Pooling node types into a single global ranking conflates distinct base rates and impact distributions, shifting the resulting rank correlation outside the envelope of per-type correlations (Section 7.3). We therefore report stratified, single-population metrics throughout and explicitly identify any pooled figures.

6.3.2. Evaluation Protocols

- **In-Distribution Evaluation:** 60% train / 20% validation / 20% test node splits over five seeds $\{42, 123, 456, 789, 2024\}$. Each split is a deterministic function of node identity *and* seed, so a seed redraws the partition as well as the initialisation. The consequence for Table 6 is that dispersion across seeds is split noise on 10–60 held-out nodes for every variant — including the training-free baselines, whose scores are otherwise deterministic — and additionally training noise for the four learned variants. Per-seed standard deviations are reported in that table for this reason.

- **Inductive Leave-One-Scenario-Out (LOSO):** Models are trained on seven synthetic scenarios and evaluated zero-shot on the held-out scenario across eight folds (including the ATM topology), testing zero-shot generalization across distinct architectural domains. Two protocol details of the harness bear on reproducibility. The largest scenario in each fold's training set is designated the *primary* graph and supplies the validation mask used for early stopping, with the remaining six passed as additional inductive graphs; this is Enterprise in seven folds and IoT Smart City in the fold that holds Enterprise out. Message-passing depth is set from the primary graph's size (one layer at $|V| \leq 200$, two at $|V| \leq 500$, otherwise three), so the Enterprise-holdout fold trains a two-layer model while the other seven train three-layer models. Both behaviours are fixed in the released harness and apply identically to every variant.

- **Real-World Architectural Transfer:** Evaluating models trained on synthetic corpora zero-shot on authentic open-source distributed systems without fine-tuning.

7. Results and Empirical Analysis

7.1. RQ1: Graph Learning vs. Structural Baselines

Table 6 presents the in-distribution held-out Spearman rank correlation (ρ) against simulated cascade impact $I^*(v)$ across all seven synthetic scenarios.

Table 6: In-distribution held-out Spearman rank correlation (ρ) against $I^*(v)$, reported as the mean over five seeds ± the standard deviation across those seeds; n is the held-out Application count. All graph learning models (GL, GL-QoS, HGL, HGL-QoS) are evaluated under substrate parity on the native multigraph; topological baselines operate on the flow projection. Each seed redraws the 60/20/20 split as well as the initialisation (Section 6.3). Read alongside the paired tests of Table 7, which use the scenario as the unit of analysis.

Scenario	n	Topo-BL	Topo-QoS	GL	GL-QoS	HGL	HGL-QoS
AV System	16	0.297 ±0.366	0.723 ±0.252	0.760 ±0.155	0.698 ±0.244	**0.789** ±0.124	0.649 ±0.121
Enterprise	60	0.415 ±0.116	0.805 ±0.059	0.579 ±0.555	0.886 ±0.048	0.880 ±0.049	**0.891** ±0.037
Financial Trading	12	0.268 ±0.341	0.700 ±0.177	0.812 ±0.132	0.821 ±0.211	**0.854** ±0.088	0.770 ±0.305
Healthcare	10	−0.155 ±0.245	**0.768** ±0.200	0.587 ±0.463	0.594 ±0.347	0.652 ±0.246	0.645 ±0.391
Hub-and-Spoke	14	0.285 ±0.257	0.401 ±0.318	**0.541** ±0.196	0.331 ±0.424	0.534 ±0.031	0.297 ±0.601
IoT Smart City	40	−0.058 ±0.152	0.071 ±0.179	0.806 ±0.068	0.603 ±0.343	**0.881** ±0.033	0.842 ±0.088
Microservices	18	0.352 ±0.348	**0.697** ±0.216	0.536 ±0.250	0.479 ±0.310	0.483 ±0.335	0.476 ±0.351
Mean	—	**0.201**	**0.595**	**0.660**	**0.630**	**0.725**	**0.653**

Table 7: Paired Wilcoxon signed-rank tests across scenarios ($n = 7$, two-sided). At $n = 7$ the smallest attainable two-sided p is 0.0156, which is above $\alpha/6 = 0.0083$; no entry in this table can therefore survive a family-wise correction over its six comparisons, and flagged results should be read as uncorrected and exploratory (Section 6.3).

Comparison	$\Delta\rho$	Won	Wilcoxon W	p-value	Significance
HGL vs. Topo-BL	**+0.524**	7/7	0.0	**0.0156**	**Statistically Significant** ($p < 0.05$)
HGL vs. GL-QoS	**+0.094**	6/7	2.0	**0.0469**	**Statistically Significant** ($p < 0.05$)
HGL vs. Topo-QoS	**+0.130**	5/7	9.0	0.469	Not significant
GL vs. Topo-QoS	**+0.065**	4/7	13.0	0.938	Not significant
HGL vs. GL	**+0.065**	5/7	5.0	0.156	Not significant
HGL-QoS vs. HGL	**−0.072**	1/7	3.0	0.078	Not significant (marginal; HGL-QoS trails in-distribution)

7.1.1. Out-of-Distribution (LOSO) Generalization

In inductive Leave-One-Scenario-Out (LOSO) cross-validation, models are evaluated on their capacity to predict cascading criticality over completely unseen system topologies:

Table 8: Inductive Leave-One-Scenario-Out (LOSO) evaluation, Application population, eight folds. Rows are grouped by role: training-free structural baselines, proposed learned predictors, and the RM/$Q(v)$ diagnostic reference of Section 1.2. Each fold score is the mean over the five seeds; **Fold** σ is the population standard deviation of those eight fold means, **Seed** σ is the median within-fold standard deviation across seeds (zero by construction for the deterministic baselines), and the 95% interval is a non-parametric bootstrap over folds ($B = 2{,}000$).

Predictor / Reference	Mean LOSO ρ	95% CI	Fold σ	Seed σ	Critical-Set $F_1@K$	Requires Training
Training-free structural baselines						
Topo-BL	0.301	[0.210, 0.379]	0.126	—	0.363	No
Topo-QoS	0.571	[0.440, 0.688]	0.181	—	0.380	No
Learned predictors						
GL (Homogeneous)	0.086	[0.007, 0.179]	0.122	0.180	0.237	Yes
GL-QoS (Homogeneous)	0.363	[0.299, 0.421]	**0.089**	0.326	0.341	Yes
HGL (Typed Heterogeneous)	0.439	[0.331, 0.530]	0.145	0.284	0.327	Yes
HGL-QoS (Typed + QoS)	**0.608**	[0.508, 0.702]	0.143	**0.083**	**0.414**	Yes
Diagnostic reference — not a ranking model						
RM / $Q(v)$	0.195	[0.102, 0.279]	0.130	—	0.327	No

Eight LOSO folds are reported, comprising the seven synthetic scenarios and the ATM case study from Section 7.3. In each fold, one scenario is held out for zero-shot testing while the model is trained exclusively on the remaining seven. All variants are evaluated on the identical Application node set per fold (Section 6.3); paired Wilcoxon tests are conducted across the eight folds. Per-fold evaluated populations range from 26 to 300 Application nodes, so $K = \text{round}(0.20\,|V_{\text{app}}|)$ ranges from 5 to 60. On $F_1@K$, HGL-QoS beats Topo-QoS in 4 of 8 folds ($W = 9.0$, $p = 0.469$) and GL in 8 of 8 ($W = 0.0$, $p = 0.0078$): the learned model separates

itself from untyped learning on this metric, but not from the training-free QoS baseline.

Label-noise ceiling. These correlations are bounded by the reproducibility of the target they are scored against. Re-running the ground-truth oracle across the five seeds gives a test–retest rank correlation of 0.97–1.00 in six of the seven synthetic scenarios; the exception is Microservices at $\rho = 0.807$, which sets the effective ceiling for cross-fold means. HGL-QoS's $\rho = 0.608$ therefore recovers roughly three quarters of the attainable signal, and no predictor in Table 8 can exceed the reproducibility of its own labels. Top-K critical sets are the noisier construct: their cross-seed Jaccard falls to 0.44 (Microservices) and 0.71 (AV, Financial Trading), which is part of why the $F_1@K$ margins above are less stable than the ranking margins.

Key Insights for RQ1:

1. **The typed predictor is the best learned configuration overall.** HGL-QoS leads every other learned variant on both metrics ($\rho = 0.608$, $F_1@K = 0.414$), with a ranking margin over both homogeneous GNNs that excludes zero (+0.246 over GL-QoS, 95% CI $[+0.158, +0.329]$; +0.522 over GL; 8/8 folds, $p = 0.0078$ in both cases).
2. **Critical-set identification separates learned models from each other, but not from the untrained QoS baseline.** HGL-QoS improves $F_1@K$ over GL by +0.177, but against Topo-QoS the margin is +0.034 (0.414 vs. 0.380), won in 4 of 8 folds with per-fold differences alternating in sign ($W = 9.0$, $p = 0.469$). Critical-set identification is not a demonstrated advantage of the learned model over the QoS-weighted baseline on this corpus.
3. **QoS encoding carries the typed model out of distribution.** HGL-QoS leads HGL by $\Delta\rho = +0.169$, winning 7 of 8 folds ($p = 0.0156$) — a substantial, consistent advantage under distribution shift.
4. **Ranking Boundary, and one significant result against us.** Against *Topo-QoS*, HGL-QoS's margin is +0.037 with a 95% bootstrap interval of $[-0.079, +0.162]$ that comfortably contains zero, won in 5 of 8 folds ($p = 0.64$). The interval, not the p-value, is the informative statement here: at this sample size the data are consistent with the learned model being anywhere from slightly worse to moderately better than the baseline. On out-of-distribution ranking, heterogeneous graph learning matches rather than outperforms a well-constructed QoS baseline, and the critical-set margin above is likewise not significant. The comparison is worse for the typed model without QoS features: HGL *trails* Topo-QoS by $\Delta\rho = -0.132$ (95% CI $[-0.220, -0.042]$, excluding zero), losing 7 of the 8 folds ($W = 3.0$, $p = 0.039$). This is the only comparison between a proposed variant and a training-free baseline in Table 8 that reaches significance in either direction, and it reaches it against us. Read together with the QoS ablation of Section 7.3, this locates the entire out-of-distribution case for the learned model in the explicit QoS encoding rather than in typed message passing alone. The case for training a model therefore rests on the qualitative capabilities the baseline cannot supply — actionable explanations, per-relationship edge criticality, and multi-task quality attributions — which this study describes but does not yet quantify (Section 8.4).
5. **The explanation layer is weakly predictive, not noise.** RM/$Q(v)$ achieves $\rho = 0.195$ under distribution shift — outperforming untyped GL (0.086) while providing interpretable architectural diagnostics (Section 5).

Figure 3 provides an overview of these results, summarizing out-of-distribution rank correlation, critical-set identification, and cross-oracle agreement.

7.2. RQ2: Value of Typed Heterogeneity

To evaluate the specific contribution of node and edge typing, we contrast relation-specific Heterogeneous Graph Transformers (HGL / HGL-QoS) against homogeneous Graph Attention Networks (GL / GL-QoS) on the shared native multigraph substrate under complete substrate parity:

- **In-Distribution Fitting (Table 6):** On familiar architectures, homogeneous and heterogeneous models exhibit comparable fitting capacity. Unweighted HGL leads unweighted GL by $\Delta\rho = +0.065$ (0.725 vs. 0.660; $W = 5.0$, $p = 0.156$, 5/7 scenarios won). When QoS edge features are encoded, the models are statistically indistinguishable: HGL-QoS achieves $\rho = 0.653$ versus GL-QoS's 0.630 ($\Delta\rho = +0.022$, $W = 13.0$, $p = 0.938$, won in 3/7 scenarios). On known topologies, homogeneous GNNs

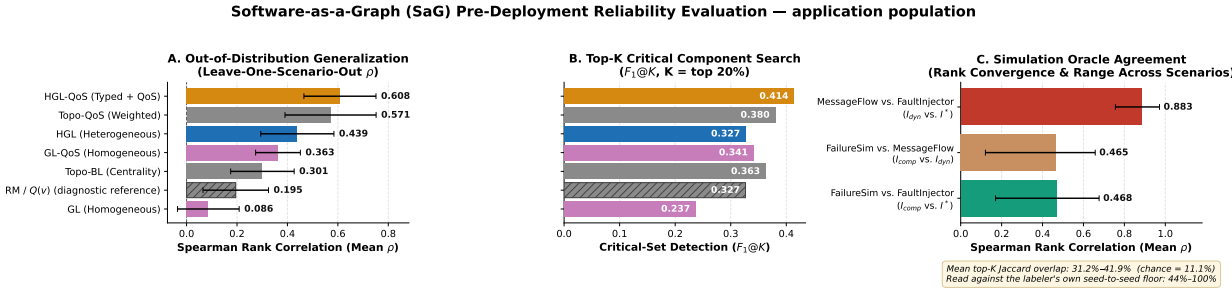

Figure 3: Results at a glance, evaluated on the Application population. **(A)** Out-of-distribution rank correlation per variant across eight LOSO folds (whiskers denote σ). **(B)** Critical-set identification at $K = 20\%$. **(C)** Pairwise agreement across the three simulation oracles.

partially compensate for missing relation types by leveraging per-type input embeddings and structural degree signatures.

- **Out-of-Distribution Generalization (LOSO, Table 8):** Under inductive distribution shift to completely unseen architectures, homogeneous models collapse while typed heterogeneous learning generalizes robustly. Heterogeneous HGL outperforms homogeneous GL by +0.353 ($\rho = 0.439$ vs. 0.086, winning *all eight* folds, $W = 0.0$, $p = 0.0078$). With QoS features enabled, HGL-QoS outperforms GL-QoS by +0.246 (0.608 vs. 0.363, $W = 0.0$, $p = 0.0078$). Without relation typing, untyped models degrade below even unweighted structural baselines (0.086 vs. 0.301).

Relational Typing as an Inductive Generalization Bias. Read together, the in-distribution and out-of-distribution results reveal a fundamental principle: *the demonstrated value of typed message passing is an inductive generalization property rather than an in-distribution fitting advantage.* When trained and evaluated on the same system distribution, both homogeneous and heterogeneous architectures have sufficient parameter capacity to fit cascading criticality patterns. However, structural degree distributions and local neighbor counts fail to transfer zero-shot across distinct architectural topologies. In contrast, relation-specific parameters (e.g., distinguishing `PUBLISHES_TO` message dissemination from `RUNS_ON` host placement) capture invariant causal semantics of failure propagation that persist across previously uncharacterized systems.

7.2.1. Empirical Edge-Removal Analysis

We further evaluated edge criticality by simulating the removal of individual edges while keeping both endpoint components operational. The 50 highest-ranked candidate edges in the `av_system` topology comprised 35 `RUNS_ON`, 11 `CONNECTS_TO`, 3 `SUBSCRIBES_TO` and 1 `PUBLISHES_TO` edge. Exactly 4 removals produced non-zero downstream cascade impact, and they were exactly the 4 communication channels: every host-placement and inter-host network edge in the pool was inert, while every publish–subscribe edge in it was not. The effect is nonetheless small in magnitude — the largest single-edge impact was 0.00504 — so the finding is that severing a message channel degrades reachability slightly whereas relocating a component or dropping a host link does not, not that any one channel is load-bearing. With only 4 communication edges in the candidate pool this is a descriptive observation on one topology rather than a tested claim, and we report it as such.

7.3. RQ3: Ablations and Sensitivity Analysis

Role of ρ in Sensitivity Sweeps. The parameter sweeps below utilize RM's rank correlation against $I^*(v)$ strictly as a *sensitivity probe* to quantify parameter leverage relative to between-model margins.

7.3.1. QoS Feature Encoding

Explicitly encoding continuous QoS attributes (HGL-QoS) trades minor in-distribution accuracy for substantial out-of-distribution robustness.

In-distribution, HGL-QoS trails base HGL slightly ($\Delta\rho = -0.072$, not statistically significant), concentrated in two atypical topologies (Hub-and-Spoke and AV). Under LOSO, this relationship reverses decisively: HGL-QoS leads by $+0.169$ ($p = 0.016$, 7/8 folds won). While structural degree signatures do not transfer across unseen architectures, declared QoS contracts preserve identical semantics (`RELIABLE` and `PERSISTENT`) everywhere.

Does the QoS gain depend on QoS diversity in the held-out system? Section 6.1 records that four of the seven synthetic scenarios carry near-constant QoS ($\sigma(w(t)) \approx 0.02$), which raises the question of whether the encoding is carrying middleware semantics or merely acting as an additional regularizing channel. Splitting the per-fold $\Delta\rho$ by the held-out scenario's QoS diversity does not support the second reading, but it does not cleanly confirm the first either. The gain is present in seven of the eight folds regardless of holdout: AV $+0.310$, IoT $+0.271$, Financial $+0.176$, Healthcare $+0.169$, ATM $+0.168$, Microservices $+0.147$, Hub-and-Spoke $+0.116$, Enterprise -0.004. Grouped, the mean gain is $+0.205$ on the three QoS-diverse holdouts and $+0.142$ on the four QoS-flat ones — larger where the held-out system has genuine QoS variation, as a semantic reading predicts, but far from absent where it does not. Because the training set in every fold mixes flat and diverse scenarios, this split bounds only the holdout side of the question; separating the two readings properly requires a corpus with QoS diversity varied independently of topology, which we do not have.

A second, unclaimed effect: seed stability. The QoS encoding also sharply reduces sensitivity to initialisation. Median within-fold standard deviation across the five seeds is 0.083 for HGL-QoS against 0.284 for HGL, 0.326 for GL-QoS and 0.180 for GL (Table 8). On the AV fold the spread narrows from ± 0.490 to ± 0.053. We report this because it bears on deployability — a gate whose verdict moves by $\pm 0.5\,\rho$ with the training seed is not one a team can act on — but we did not predict it, it is measured on eight folds at five seeds, and we do not offer a mechanism for it.

Enterprise is the one fold without a gain, and it is also the one confounded fold. The single non-positive fold (-0.004) is Enterprise, which is also the only fold whose model is trained at two message-passing layers rather than three, because harness depth is set from the primary training graph's size (Section 6.3). Depth is therefore confounded with fold identity at exactly the point where the effect disappears. We cannot separate the two from the runs reported here, and we flag it rather than reading the Enterprise result either way.

7.3.2. Topic-Weight Coefficients (β, α, ψ)

We swept (β, α, ψ) over a grid spanning the entire simplex to evaluate sensitivity to topic coupling weights, with results detailed in Table 9:

Table 9: Sensitivity of topic-weight ordering and downstream rank correlation against $I^*(v)$ to coefficients (β, α, ψ) (Application population, mean over 7 scenarios, 375 topics).

(β, α, ψ)	ρ **of** $w(t)$ **vs. shipped**	**Topo-QoS** ρ	**RM** ρ
$(0.75, 0.15, 0.10)$ *shipped*	1.000	0.604	0.267
$(0.90, 0.05, 0.05)$	0.966	0.600	0.273
$(0.85, 0.15, 0.00)$	0.950	0.608	0.268
$(1.00, 0.00, 0.00)$	0.852	0.618	0.268
$(0.60, 0.20, 0.20)$	0.970	0.604	0.261
$(0.50, 0.25, 0.25)$	0.942	0.603	0.259
$(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ *uniform*	0.870	0.608	0.256
Spread over grid	—	**0.018**	**0.017**

Topic-weight coefficients are not load-bearing: the ordering of $w(t)$ never falls below $\rho = 0.852$ against the default, and downstream rank correlations vary by at most 0.018 (Topo-QoS) and 0.017 (RM).

7.3.3. Intra-Dimension AHP Weight Shrinkage

We evaluated the sensitivity of the RM attribution baseline across shrinkage parameter $\lambda \in [0, 1]$, blending internal term weights from a uniform prior ($\lambda = 0$) to raw AHP judgment ($\lambda = 1$), as summarized in Table 10:

Table 10: Sensitivity of RM rank correlation against $I^*(v)$ to AHP shrinkage parameter λ (Application population, mean over 7 scenarios).

λ **Setting**	**0.00 (Uniform)**	**0.50**	**0.70 (Default)**	**0.80**	**1.00 (Raw AHP)**
Mean Rank Correlation (ρ)	**0.348**	0.291	0.267	0.256	0.232

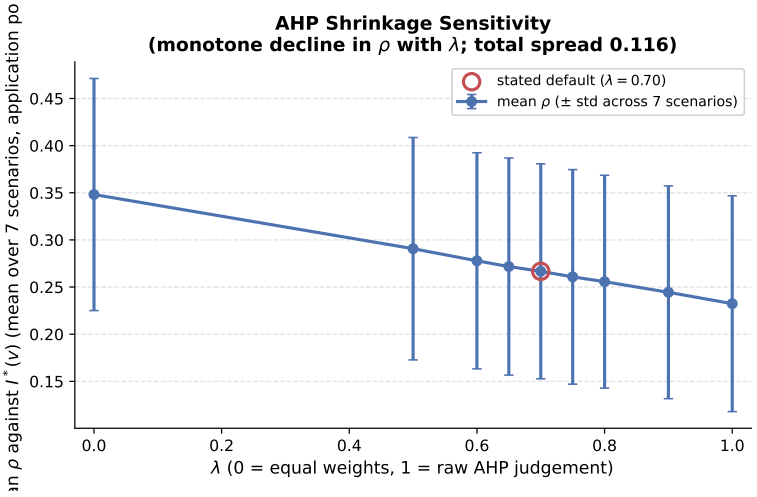

Figure 4: Sensitivity of RM composite rank correlation against $I^*(v)$ across AHP shrinkage λ (Application population, mean over 7 scenarios).

Rank correlation decreases monotonically as weights transition from uniform toward raw AHP ($\rho = 0.348 \rightarrow 0.232$, Figure 4). Elicited AHP weights provide transparent, auditable domain attribution rather than optimizing rank correlation. We retain $\lambda = 0.70$ on that basis.

7.3.4. Joint Sensitivity Across All Ten Weight Constants

To assess interactions, we swept all ten weight constants jointly across six scenarios using Morris elementary-effects screening [79, 80] (a computationally efficient alternative to variance-based global sensitivity analysis [81, 82]), with factor influences reported in Table 11:
Morris screening confirms that λ and r_α are the primary drivers, while topic and QoS sub-weights reside in a modest band ($\mu^* \in [0.012, 0.019]$). Dirichlet simplex sampling over 100 draws confirms tight stability: mean $\rho = 0.243$ (standard deviation 0.006, 90% interval $[0.231, 0.253]$, mean top-20% Jaccard 0.825).

7.3.5. Convergent Validity Over Simulation Oracles

We evaluated inter-oracle agreement across $I^*(v)$ (`FaultInjector`), $I_{\text{comp}}(v)$ (`FailureSimulator`), and $I_{\text{dyn}}(v)$ (`MessageFlowSimulator`) over seven scenarios, summarized in Table 12:
Ordering converges strongly between the queue-flow simulator and topological cascade oracle ($\rho = 0.883$). Top-K set agreement is more conservative (Jaccard 0.42), bounded by discrete threshold sensitivity in non-linear cascades.

7.3.6. Domain-Specific Weighting and Threshold Sensitivity

Sweeping composite reliability weight $w_R \in [0, 1]$ moves mean ρ by only 0.024 ($0.341 \rightarrow 0.365$), with mean Kendall correlation $\tau = 0.974$ between domain and static rankings. Within that narrow band the

Table 11: Morris elementary-effects screening [79, 80] across ten weight constants, ranked by influence (μ^*) on mean ρ (6 scenarios, 10 trajectories, 110 evaluations).

Factor	μ^*	σ
λ (AHP shrinkage)	0.124	0.077
r_α	0.096	0.035
α	0.019	0.034
w_{rel}	0.019	0.018
β	0.017	0.023
w_{dur}	0.014	0.017
w_{prio}	0.013	0.018
ψ	0.012	0.015
p (power-mean exponent)	0.005	0.005
γ (fan-out)	0.001	0.002

Table 12: Inter-oracle agreement across simulation paradigms (chance top-K Jaccard is 0.111).

Oracle pair	**Mean ρ (range)**	**Mean τ**	**Jaccard@K**	**Tie-robust**
I_{dyn} vs. I^*	**0.883** (0.756–0.972)	0.745	0.424	0.419
I_{comp} vs. I^*	0.468 (0.171–0.677)	0.349	0.307	0.312
I_{comp} vs. I_{dyn}	0.465 (0.121–0.658)	0.348	0.313	0.313

domain-derived weighting does not improve on the alternatives it replaces — mean ρ is 0.347 (domain-derived), 0.349 (equal), and 0.353 (static) — so domain reweighting should be understood as an attributional device that expresses criticality in stakeholder terms, not as a ranking-accuracy mechanism. No weighting confined to this parameter could move ρ appreciably. Sweeping cascade propagation thresholds revealed higher sensitivity ($\Delta\rho = 0.102$), with ranking performance plateauing above 0.35.

7.3.7. Anti-Pattern Detection and Node-Type Stratification

Validated against $I_{\text{comp}}(v)$, the rule-based anti-pattern catalog reaches mean precision 0.239 and recall 0.900 ($F_1 = 0.378$), but the recall must be read against a flag rate of 94.2%: the 18 active detectors implicate almost everything, so recall near 0.9 is close to what indiscriminate flagging yields and precision 0.239 is roughly the base rate. Chance-corrected agreement confirms this and worsens with scale — over the ATM sweep (29 to 444 components, five seeds) Cohen's κ falls monotonically from 0.118 to -0.045, i.e. to no better than chance on the largest graph. We report the catalog as a coarse, deliberately over-inclusive triage filter, *not* a validated detector; every quantitative critical-set claim in this paper rests on the ranking models of Sections 7.1–7.2 instead. A nineteenth detector, `DEEP_PIPELINE`, is excluded outright: enumerating every simple source-to-sink path, it emitted 247,761 findings without terminating on a 29-component fixture.

Stratification vs. Pooling: Measured against the composite oracle $I_{\text{comp}}(v)$ over the eight scenarios of the detection benchmark, stratified RM rank correlations are $\rho = 0.503$ (Application, 8 scenarios), 0.395 (Broker, 6 scenarios), and 0.142 (Node, 8 scenarios), while pooled correlation across all types is $\rho = 0.028$. These figures are not comparable to the RM values elsewhere in this paper and should not be read as the same quantity: Table 10's $\rho = 0.267$ scores RM in-sample against $I^*(v)$ over the seven synthetic scenarios, and Table 8's $\rho = 0.195$ scores it zero-shot against $I^*(v)$ over the eight LOSO folds. Oracle, corpus and protocol all differ; the contrast drawn here is strictly between the stratified and pooled readings of one benchmark. Pooling node types triggers Simpson's paradox by mixing distinct base rates. Classifying components within node types changes the criticality tier of 62.8% of components (with 19.0% crossing the critical boundary), confirming that evaluation and gating must operate strictly per-type. On that same pooled benchmark, unweighted degree centrality attains $\rho = 0.166$ and $F_1 = 0.413$ against RM's 0.028 and 0.349 — a further reason to read $Q(v)$ as an attribution instrument rather than as a ranking model.

7.3.8. HGT Attention Weight Analysis

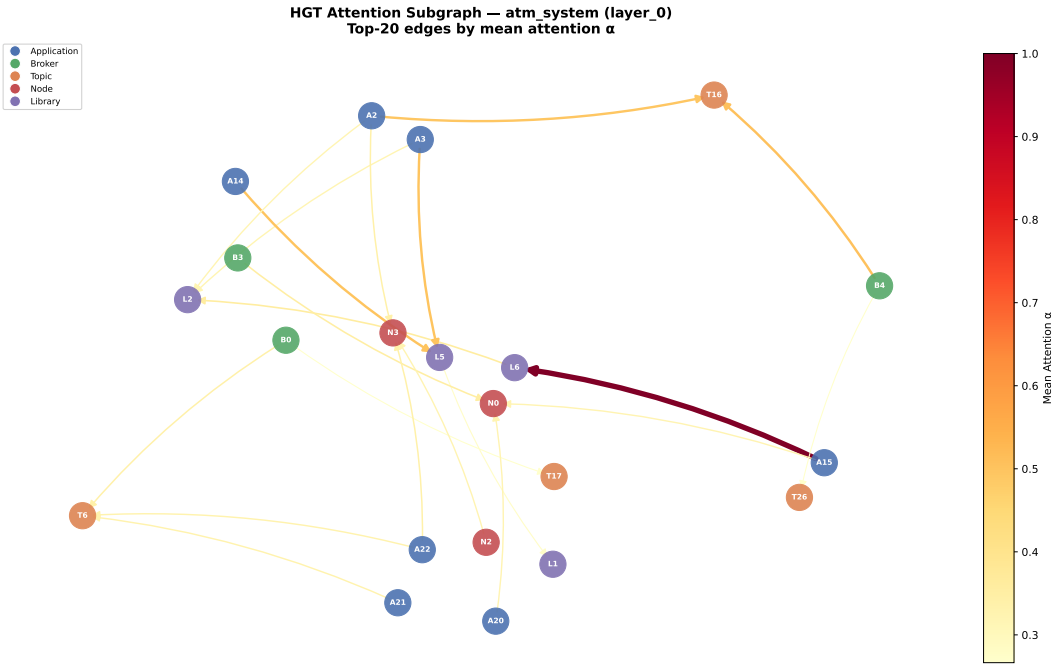

Figure 5: Relational attention extracted from the trained HGT on the ATM case study. Peak attention focuses on `USES` (Application → Library) and `ROUTES` (Broker → Topic) channels.

Aggregated by relation type over the ATM case study, first-layer mean attention orders `USES` into libraries (0.227, 0.215) above `ROUTES` (0.194), `SUBSCRIBES_TO` (0.176), `PUBLISHES_TO` (0.163) and `RUNS_ON` (0.153), with the same ordering to within 0.02 in layers two and three. Two cautions bound what this supports. The spread across all eight relation types is narrow (0.15–0.23), so this is a tendency, not a separation; and because α is a softmax over each destination's incoming edges, mean α is driven substantially by in-degree — the top-ranked `Library`→`Library` relation carries 4 edges, and the largest weight in the graph ($\alpha_{uv} = 1.00$) sits at a destination of in-degree one, where $\alpha = 1$ holds by construction. Figure 5 is therefore a qualitative illustration on one topology at one seed without a significance test, not evidence for why typing helps.

7.4. RQ4: Real-World Distributed Architecture Validation

We evaluated SaG zero-shot on three open-source distributed systems transcribed from their public repositories by dedicated architectural adapters, with results presented in Table 13. We note the scope of this evaluation before reporting it. Online Boutique is a vendor demonstration application and Train-Ticket an academic benchmark, so these are reference implementations rather than production deployments; each is an abstraction of its upstream repository rather than a complete transcription; and their labels come from the same simulation oracle used throughout, so what follows tests topological transfer, not agreement with observed field failures. Two further limits bound what this table can support. First, only the typed variant (HGL) was evaluated here: no homogeneous or QoS-weighted baseline was run on these systems, so the typing claim of Section 7.2 is *not* tested outside our own generator anywhere in this study, and should be read as a synthetic-corpus result until a matched comparison on non-generated architectures is available. Second, the labels are strongly tied at zero — 13 of 32 Applications in Autoware, 14 of 22 in Cloud Microservices and 27 of 41 in Train-Ticket carry zero simulated impact (the complement of the "Impactful Apps" column). Spearman ρ over a population that is 41–64% tied is dominated by how those ties are handled; the values below use midrank ties throughout, and the highest correlation in the table ($\rho = 0.778$) rests on the population with the largest tied block, so it is the least, not the most, informative entry.

Table 13: Zero-shot transfer evaluation on open-source reference systems (held-out, without fine-tuning). The evaluated variant is HGL (typed, QoS edge features disabled); ± denotes the standard deviation over the five training seeds $\{42, 123, 456, 789, 2024\}$. "Gain vs. Deg" is $|\rho_{\text{SaG}}| - |\rho_{\text{deg}}|$. Ground truth is simulator-derived, as elsewhere in this study; these are not observed production incidents.

Real-World Architecture	$\lvert V\rvert$	$\lvert V_{\text{app}}\rvert$	Spearman ρ	Kendall τ	App $F_1@K$	Pooled $F_1@K$	Impactful Apps	Gain vs. Deg
Autoware.universe (ROS 2)	75	32	**0.685 ± 0.010**	0.513	0.333	0.800	19 / 32	+0.357
Cloud Microservices Mesh	60	22	**0.778 ± 0.001**	0.639	0.500	1.000	8 / 22	+0.014
Train-Ticket Booking Mesh	90	41	**0.759 ± 0.001**	0.605	0.625	1.000	14 / 41	+0.264

Key Insights for RQ4:

1. **Strong Real-World Zero-Shot Transfer:** Trained exclusively on synthetic scenarios, SaG achieves high zero-shot rank correlation on Cloud Microservices ($\rho = 0.778$), Train-Ticket ($\rho = 0.759$), and Autoware.universe ($\rho = 0.685$), indicating that relation-specific graph representations transfer to architectures outside the generator's distribution. Because the comparison in this table is against degree centrality alone, it does not establish transfer superiority over the stronger `Topo-QoS` baseline of Table 8.
2. **Stratified vs. Pooled Critical Set Identification:** On the stratified Application population (V_{app} with $K = \text{round}(0.20 \cdot |V_{\text{app}}|)$), SaG identifies the top-20% critical services with $F_1@K = 0.333$ in Autoware, 0.500 in Cloud Microservices, and 0.625 in Train-Ticket. When evaluated across the pooled multigraph ($K = \text{round}(0.20 \cdot |V|)$), the pooled $F_1@K$ reaches 0.800, 1.000, and 1.000, respectively. This disparity highlights the base-rate phenomenon discussed in Section 6.3: unmeasured passive infrastructure entities (Topics, Nodes, Libraries) exhibit constant zero failure impact under process crash simulation. While the pooled metric confirms that SaG cleanly separates critical active services from inert infrastructure, reporting stratified V_{app} metrics provides the more rigorous, non-inflated evaluation.
3. **The framework's own acceptance gates do not pass on any of the three systems.** SaG ships a five-condition release gate ($\rho \geq 0.75$, $F_1 \geq 0.65$, SPOF $F_1 \geq 0.60$, fault-tolerance ratio ≤ 0.30, prediction gain ≥ 0.02). Overall pass rate is zero on all three architectures: Autoware fails the ρ and SPOF conditions, Cloud Microservices fails SPOF and prediction gain, and Train-Ticket fails SPOF. The SPOF condition fails everywhere ($F_1 = 0.50, 0.33, 0.57$), which matters disproportionately because single-point-of-failure attribution is the Availability half of the explanation layer's claim in Section 5 and this is the only place in the study where that claim is tested directly against a system we did not generate. We report the gate outcomes rather than the ranking metrics alone because the gap between them is itself the finding: rank ordering transfers to these architectures, whereas the absolute, threshold-based judgements the framework would issue in CI/CD do not yet clear their own bars.
4. **Predictive Advantage over Structural Heuristics:** The "Gain vs. Deg" column reports SaG's rank-correlation margin over an unweighted degree centrality baseline (+0.357 on Autoware, +0.264 on Train-Ticket, +0.014 on Cloud Microservices), consistent with heterogeneous message passing capturing multi-hop topic–broker cascades that degree centrality does not resolve. This margin is a difference between aggregate rank correlations and should not be read as a significance result: the paired per-node test recorded alongside it — a Wilcoxon signed-rank test on $|\hat{Q}(v) - I^*(v)| - |\deg(v) - I^*(v)|$ — does not reach significance on any of the three systems ($p \geq 0.97$). That test compares raw score errors between quantities on different scales and is therefore weak evidence in either direction, but we report it rather than omit it. Evaluating `Topo-QoS` on these authentic systems was precluded because public open-source manifests lack declared fine-grained QoS contracts (such as durability and transport priorities); without synthetic QoS imputation, only unweighted structural degree could be evaluated directly, bounding this real-world evaluation to outperforming unweighted structural heuristics. Establishing a transfer advantage over QoS-weighted baselines on real-world systems remains an objective for future QoS-annotated benchmarks.

7.5. RQ5: Analysis Cost, Computational Sustainability, and CI/CD Feasibility

RQ5 quantifies computational overhead and sustainability during CI/CD evaluation, with per-stage latencies reported in Table 14:

Table 14: Per-stage latency of the inference pipeline across scaling graph sizes (CPU, median of 3 runs).

$\|V\|$	$\|E\|$	**Analyse (s)**	**Graph→tensor (s)**	**HGT forward (ms)**	**Analyse : forward**
249	1,127	0.27	0.011	22.5	12×
499	2,402	0.95	0.022	15.7	61×
999	6,422	4.74	0.055	36.7	129×
1,998	19,301	23.83	0.153	43.7	**545**×

The neural model is the cheapest stage, but not the whole cost. At 2,000 components the HGT forward pass takes 43.7 ms against 23.8 s for deterministic structural analysis. That ratio locates cost *inside* the pipeline; it is not the cost of evaluating an architecture. Indices 0–17 of every node feature vector (Section 3.4) — betweenness, closeness, reverse PageRank, articulation and bridge scores — are products of that same analysis stage, so the forward pass cannot run without it. End-to-end evaluation of an unseen 2,000-component architecture therefore costs about 24 s, of which the model is 0.2%; the 43.7 ms figure is the marginal cost of re-scoring an already-analysed graph. Cost scales with edge density rather than node count: the 520-component Enterprise mesh needs 27.2 s for its 3,245 structural edges, while a sparser 999-component system needs 4.7 s. Across the corpus the complete gate (structural analysis plus 18 anti-pattern detectors) runs in 0.02–27.4 s. Since the dominant stage is $O(|V|^2 + |V||E|)$, this budget should not be extrapolated: we have measured up to 2,000 components, and the systems invoked in Section 1.1 can be an order of magnitude larger.

The quantity measured is wall-clock time, against multi-seed discrete-event simulation sweeps that are substantially more expensive on the same corpus. We deliberately do not convert it into an energy or carbon figure (Section 8.2). What the measurements support is the narrow claim that pre-deployment gating fits inside a pull-request budget at the scales we tested, without a simulation cluster.

8. Discussion, Threats to Validity, and Conclusion

8.1. Discussion and Practical Implications

Role of the Predictive Pathway. Heterogeneous graph learning proves superior in three critical aspects: (1) *Zero-Shot Inductive Generalization:* It is the most effective *learned* model on unfamiliar architectures ($\rho = 0.608$ out-of-distribution); at matched QoS encoding the typed model leads its untyped counterpart by +0.246, and by +0.353 when neither encodes QoS, in both cases winning all eight folds ($p = 0.0078$; Section 7.2). (2) *Shortlist Identification:* It attains the highest critical-set score of any variant we evaluated ($F_1@K = 0.414$), though its margin over the training-free QoS baseline is small and not statistically significant (4/8 folds, $p = 0.47$; Section 7.1). (3) *Low Marginal Cost:* Re-scoring an analysed graph costs 43.7 ms, and the complete gate including feature computation runs in 0.02–27.4 s — fast enough for pull-request gating rather than deferred nightly batch sweeps (Section 7.5). Beyond scalar rankings, HGT provides relation-specific attention over the channels driving cascade spread, per-relationship edge criticality, and multi-task quality attributions that degree centralities cannot produce.

Boundary Conditions and When Not to Train. The honest comparison is against a training-free QoS-weighted centrality baseline (`Topo-QoS`), where the out-of-distribution ranking margin is +0.037 and not statistically significant ($p = 0.64$, Table 8). For engineering teams requiring only a component ranking on architectures similar to those already characterized, `Topo-QoS` is recommended: it requires no training, corpus, or model checkpoints. Graph learning is justified when explanatory attributions, per-relationship criticalities, or inductive zero-shot transfer across structurally distinct architectures are required. Between the two typed variants, HGL provides higher in-distribution fidelity, while HGL-QoS provides essential robustness under distribution shift (Section 7.3).

Role of the Explanation Layer. The RM profile separates single-point-of-failure exposure (Availability) from wide error propagation (Fault Tolerance), a distinction neither the predictor nor the oracle produces and one that decides between replication and circuit-breaking. Table 8 and Figure 3 therefore list RM alongside the ranking predictors as a diagnostic reference, not a competitor. Its practical value rests on that distinction being correct and useful, which this study specifies but does not evaluate against expert judgement or remediation outcomes.

8.2. Performance and Computational Sustainability Implications

The cost profile in Section 7.5 inverts the usual intuition about deep learning in continuous delivery: within SaG the neural model is the cheapest stage, and its margin widens with scale (12× at 249 nodes, 545× at 1,998). Because feature extraction is a prerequisite rather than an alternative, the figure that matters operationally is the end-to-end one, and it is small enough for per-commit gating without a simulation cluster. Two qualifications bound this. First, the deterministic stage is $O(|V|^2 + |V||E|)$, so the corpus measurements do not extrapolate to the very large deployments the introduction invokes: at the observed growth, a 10^4-component system would leave the pull-request budget, and we have not measured one. Second, the quantity we measured is wall-clock time, not energy. No power counters were instrumented, the mapping from CPU-seconds to joules depends on hardware and utilisation we did not control, and a latency ratio is not an energy ratio.

On the sustainability argument. The broader claim available to a framework of this kind — that preventing cascading failures averts the retry storms, restart thrashing and connection-pool starvation that burn production capacity on uncompleted work — is a motivation for the research programme, not a result of this study. We did not deploy SaG, prevent an incident, or measure avoided compute. We state the mechanism because it is what would make the work matter for sustainability, and we mark it as unmeasured because it is.

8.3. Threats to Validity

Construct Validity. Our ground truth is generated via discrete-event cascade simulation on structural models rather than observing live production outages. To characterize construct divergence, we evaluated the three reliability-facing simulation engines (`FaultInjector`, `FailureSimulator`, `MessageFlowSimulator`) of our four-oracle taxonomy (Section 4.3). On *ordering*, the evidence converges strongly: the behavioural queue-flow simulator and the topological cascade oracle agree at $\rho = 0.883$ (never below 0.756 across scenarios), confirming that $I^*(v)$ tracks dynamic service disruption reach rather than purely static connectivity. However, queue simulation deltas are unweighted by message priority and only partially sensitive to delivery policies. On *critical-set membership*, agreement is lower: top-K Jaccard sits between 0.31 and 0.42 (against 0.111 by chance). This is not an artifact of tie-breaking (≤ 0.005 shift under tie-robust cuts), but reflects intrinsic non-linear cascade threshold sensitivity and oracle self-agreement noise (oracles achieve only 0.44 Jaccard against their own re-runs in worst-case scenarios). A further 30–47% of components per scenario carry no simulated ground truth because the cascade model cannot express direct Topic or Node failure. The fourth oracle, $I_M(v)$ (`ChangePropagationSimulator`), targets Maintainability and traverses the same derived dependency topology from which $M(v)$ is scored; agreement between them serves as an internal consistency check rather than independent validation. Settling this requires an external referent independent of topology, such as version-control code churn or historical co-change coupling, left to future work.

Internal Validity and Substrate Confound. Potential feature leakage is prevented by strict graph view separation: predictors operate exclusively on G_{analysis}, whereas ground-truth simulators operate on $G_{\text{structural}}$. Two design changes made during development bear on how the shipped formulation should be read. First, an early hard articulation-point gate caused Availability $A(v)$ to collapse to a constant multiple of $w(v)$ in six of eight scenarios where applications do not form cut vertices; replacing it with a continuous redundancy-deficit metric (fractional increase in average shortest paths upon removal) restored structural discriminability, and Equation (16) is the post-change form. That diagnosis was made on the pre-recalibration code and, unlike every measurement in Section 7, is not backed by a regenerable artifact in the replication package; we record

it as development history, not as a result. Second, we identified and resolved a normalization defect in the code-quality scoring pipeline: a min-max normalization helper previously assigned a maximal penalty to zero-variance populations, penalizing unanalyzed libraries lacking source-level `code_metrics`. Scoring undifferentiated populations with zero penalty corrected library Maintainability tiers while leaving Application rank correlations unaffected ($\Delta\rho \in \{-0.003, 0.000, 0.000\}$). In addition, four of the seven synthetic scenarios feature domain-lookup QoS tables that assign identical profiles across topics ($w(t)$ standard deviation ≈ 0.02). `Topo-QoS` nevertheless gains $+0.319\,\rho$ over `Topo-BL` in those scenarios (Table 6), a gain resulting from uniform edge rescaling rather than QoS diversity. This corpus characteristic limits what the QoS ablations of Section 7.3 can establish: on four of seven scenarios there is almost no QoS variation for the encoding to exploit. Crucially, as established in Section 6.2.1, all learned models (GL, GL-QoS, HGL, HGL-QoS) operate under complete substrate parity on the shared native multigraph in both in-distribution and out-of-distribution evaluations. This experimental parity ensures that the out-of-distribution performance margin ($\Delta\rho = +0.246$, $p = 0.0078$) strictly isolates relation-specific parameterization as an inductive generalization mechanism rather than multi-entity visibility. In-distribution, the homogeneous architecture matches the typed model when given identical multigraph visibility ($\rho = 0.630$ vs. 0.653, $p = 0.938$), demonstrating that relational typing's primary benefit is cross-architecture generalization rather than in-distribution fitting capacity. Furthermore, as disclosed in Section 8.2, sustainability claims rest on CPU latency reductions rather than direct hardware power counters (RAPL, NVML).

External Validity and Real-World Metric Stratification. While our evaluation spans ten architectures across six domains, including three authentic open-source distributed systems (ROS 2 Autoware, Cloud Microservices, Train-Ticket), future work should evaluate larger enterprise deployments with hundreds of microservices. Crucially, as shown in Table 13, evaluating top-K critical sets across the pooled multigraph yields elevated $F_1@K$ scores (0.800–1.000) because unmeasured infrastructure entities carry constant zero failure labels. Reporting stratified metrics on active application services ($F_1@K = 0.333, 0.500, 0.625$) provides the true, non-inflated operational baseline.

Conclusion Validity. Given the heavy-tailed distribution of cascading failure impacts, statistical comparisons utilize non-parametric rank correlation (Spearman ρ, Kendall τ), bootstrap confidence intervals ($B = 2,000$), and paired Wilcoxon signed-rank tests. Two critical hazards that altered conclusions in this study deserve explicit statement: (1) *Simpson's Paradox:* Pooling rank correlations across node types moves them outside the envelope of the per-type correlations they aggregate, on both benchmarks we ran. Against $I_{\text{comp}}(v)$, RM's per-type correlations of 0.503/0.395/0.142 pool to 0.028 (Section 7.3). Against $I^*(v)$ on the LOSO folds, pooling places RM at -0.014 and untyped GL at 0.381, exactly reversing the standing that stratified evaluation gives them ($+0.195$ and 0.086). The two pooled figures are separate measurements against different oracles and corpora, and are not comparable to each other. All reported metrics are therefore stratified on a single stated population (V_{app}). (2) *Substrate Conflation:* Early RM ablations scored $Q(v)$ from features restricted to the Application–Library `DEPENDS_ON` projection, where cut-vertex Availability terms vanished, producing false negative rank correlations. Computing features through the full multigraph pipeline resolved this issue.

8.4. Limitations and Future Work

Distributed AI and LLM Serving Topologies. Modern AI infrastructure relies on distributed LLM serving systems (e.g., vLLM, Triton, DeepSpeed) characterized by complex tensor and pipeline parallelism across GPU nodes, dynamic KV-cache routing, and disaggregated prefill-decode architectures. A straggler or failing node in a pipeline-parallel ring induces severe head-of-line blocking and massive GPU idle power dissipation. Extending the SaG multigraph schema to model distributed model serving topologies (nodes representing GPU workers, edges encoding tensor communication channels and KV-cache transfer fabrics) offers a promising avenue to forecast and mitigate bottlenecks in AI serving clusters.

Empirical Power and Hardware Testbeds. Validating the sustainability thesis through empirical power measurements using running package counters (Intel/AMD RAPL, NVIDIA NVML) and IPMI sensors on Kubernetes clusters during live fault injection, alongside Hardware-in-the-Loop (HIL) validation on cyber-physical testbeds (e.g., ROS 2 CAN-bus autonomous driving compute boxes).

Automated Refactoring and Self-Healing. Extending SaG from predictive analysis to prescriptive synthesis—automatically generating pull requests that reconfigure QoS policies, insert circuit-breakers, and add redundant broker pathways to eliminate single points of failure.

8.5. Conclusion

This work introduced **Software-as-a-Graph (SaG)**, a pre-deployment Static System Analysis framework that bridges the Architecture–Code Gap by combining a relation-specific Heterogeneous Graph Transformer (HGT) for failure-impact forecasting with an interpretable ISO/IEC 25010 Reliability–Maintainability explanation layer. Of the three concerns the framework is motivated by, this study delivers evidence on one and a half: **reliability**, through inductive cross-topology failure blast-radius forecasting on unseen architectures, and **analysis cost**, through a complete pre-deployment gate measured in seconds rather than cluster-hours. **Performance** and **energy** are motivating concerns we do not measure — no latency or throughput quantity is predicted, and no power counter was instrumented.

Our empirical results across synthetic systems and three open-source reference systems (ROS 2 Autoware, Cloud Microservices, Train-Ticket) show that typed learning over the full multigraph generalizes out of distribution better than untyped learning at matched QoS encoding: $\rho = 0.439$ vs. 0.086 without QoS edge features and 0.608 vs. 0.363 with them, 8 of 8 folds in both cases ($p = 0.0078$). In-distribution the two families are indistinguishable ($\rho = 0.630$–0.660 vs. 0.653–0.725), so typing is an inductive generalization property rather than a fitting advantage. Because every fold in this comparison is drawn from a single parameterized generator, and the real-world evaluation of Section 7.4 reports only the typed variant, this claim is established on synthetic architectures and remains untested on systems we did not generate. We are equally explicit about the boundary: against an unparameterized QoS-weighted centrality baseline (`Topo-QoS`), out-of-distribution ranking is matched rather than beaten ($\Delta\rho = +0.037$, $p = 0.64$), and the critical-set margin ($F_1@K = 0.414$ vs. 0.380) is not significant either (4 of 8 folds, $p = 0.47$). On present evidence a team needing only a ranking should use the baseline; the case for the learned model rests on the relational attention, per-relationship criticality and standards-grounded attribution the baseline cannot produce — specified and shown qualitatively here, and quantifying them is the natural next step. SaG establishes the typed substrate on which both prediction and explanation can be posed, providing an empirical and architectural foundation for dependable distributed systems engineering.

Declarations

CRediT Authorship Contribution Statement. FIRST-NAME LAST-NAME: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **FIRST-NAME LAST-NAME:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision, Project administration.

Declaration of Competing Interest. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data Availability. The complete replication package—including synthetic scenario datasets, generator configurations, simulation harnesses, real-world architecture adapters, trained model checkpoints, and all analysis scripts—is deposited in a public archive with a persistent identifier and cited as [83].

Declaration of Generative AI in the Writing Process. During the preparation of this work, the authors used AI-assisted language tools to check grammar, improve readability, and support LaTeX typesetting. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

[1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, Science Robotics 7 (66) (2022) eabm6074.

[2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: Proc. 6th Int. Workshop on Networking Meets Databases (NetDB), 2011.

[3] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).

[4] OASIS, MQTT version 5.0, OASIS Standard (2019).

[5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: Present and Ulterior Software Engineering, Springer, 2017, pp. 195–216.

[6] S. Newman, Building Microservices: Designing Fine-Grained Systems, O'Reilly Media, 2015.

[7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, ACM Computing Surveys 35 (2) (2003) 114–131.

[8] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, IEEE Transactions on Dependable and Secure Computing 1 (1) (2004) 11–33.

[9] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).

[10] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).

[11] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS), 1998, pp. 68–78.

[12] L. Bass, P. Clements, R. Kazman, Software Architecture in Practice, 3rd Edition, Addison-Wesley, 2012.

[13] W. Cunningham, The WyCash portfolio management system, in: Addendum to the Proc. Conf. on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA), 1992, pp. 29–30.

[14] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR), 2009, pp. 255–258.

[15] SonarSource, Clean as you code, SonarQube documentation (2024).

[16] T. J. McCabe, A complexity measure, IEEE Transactions on Software Engineering SE-2 (4) (1976) 308–320.

[17] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, IEEE Transactions on Software Engineering 20 (6) (1994) 476–493.

[18] N. Fenton, J. Bieman, Software Metrics: A Rigorous and Practical Approach, 3rd Edition, CRC Press, 2014.

[19] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, IEEE Software 33 (3) (2016) 35–41.

[20] L. C. Freeman, A set of measures of centrality based on betweenness, Sociometry 40 (1) (1977) 35–41.

[21] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, Computer Networks and ISDN Systems 30 (1–7) (1998) 107–117.

[22] U. Brandes, A faster algorithm for betweenness centrality, Journal of Mathematical Sociology 25 (2) (2001) 163–177.

[23] M. E. J. Newman, Networks: An Introduction, Oxford University Press, 2010.

[24] [REPLACE WITH FULL AUTHOR LIST], Authors' prior conference paper on multi-layer graph dependency analysis for publish–subscribe systems, pLACEHOLDER – replace with the full citation (venue, year, pages, DOI) before submission (n.d.).

[25] R. C. Cheung, A user-oriented software reliability model, IEEE Transactions on Software Engineering SE-6 (2) (1980) 118–125.

[26] K. Goseva-Popstojanova, K. S. Trivedi, Architecture-based approach to reliability assessment of software systems, Performance Evaluation 45 (2–3) (2001) 179–204.
[27] A. Immonen, E. Niemelä, Survey of reliability and availability prediction methods from the architectural perspective, Software and Systems Modeling 7 (1) (2008) 49–65.
[28] S. Becker, H. Koziolek, R. Reussner, The Palladio component model for model-driven performance prediction, Journal of Systems and Software 82 (1) (2009) 3–22.
[29] G. Franks, T. Al-Omari, M. Woodside, O. Das, S. Derisavi, Enhanced modeling and solution of layered queueing networks, IEEE Transactions on Software Engineering 35 (2) (2009) 148–161.
[30] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, C. Delimitrou, Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices, in: Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2019.
[31] Y. Gan, M. Liang, S. Dev, D. Lo, C. Delimitrou, Sage: Practical and scalable ML-driven performance debugging in microservices, in: Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2021.
[32] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS), 2020.
[33] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, D. Pei, Practical root cause localization for microservice systems via trace analysis, in: Proc. IEEE/ACM Int. Symposium on Quality of Service (IWQoS), 2021.
[34] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning, in: Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2022.
[35] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2023.
[36] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, IEEE Transactions on Software Engineering 22 (10) (1996) 751–761.
[37] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: Proc. 27th Int. Conf. on Software Engineering (ICSE), 2005, pp. 580–586.
[38] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE), 2007.
[39] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, IEEE Transactions on Software Engineering 33 (1) (2007) 2–13.
[40] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA), LNCS 5581, 2009, pp. 146–162.
[41] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, IEEE Software 35 (3) (2018) 56–62.
[42] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, Journal of Systems and Software 101 (2015) 193–220.
[43] J. Humble, D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Addison-Wesley, 2010.
[44] L. Chen, Continuous delivery: Huge benefits, but challenges too, IEEE Software 32 (2) (2015) 50–54.
[45] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).
[46] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality measure elements, Tech. rep., International Organization for Standardization (2012).
[47] T. L. Saaty, The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation, McGraw-Hill, 1980.
[48] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, Nature 406 (2000)

378–382.
[49] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, Physical Review E 66 (2002) 065102(R).
[50] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, Nature 464 (2010) 1025–1028.
[51] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, Nature Machine Intelligence 2 (2020) 317–324.
[52] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM), 2019, pp. 559–568.
[53] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track, 2024, arXiv:2402.02827.
[54] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2017.
[55] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 1024–1034.
[56] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2018.
[57] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: Proc. European Semantic Web Conference (ESWC), 2018, pp. 593–607.
[58] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: Proc. The Web Conference (WWW), 2019, pp. 2022–2032.
[59] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: Proc. The Web Conference (WWW), 2020, pp. 2704–2710.
[60] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: Proc. The Web Conference (WWW), 2020, pp. 2331–2341.
[61] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, pp. 9244–9255.
[62] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang, Parameterized explainer for graph neural network, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 19620–19631.
[63] J. Pearl, Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference, Morgan Kaufmann, 1988.
[64] G. Beliakov, A. Pradera, T. Calvo, Aggregation functions: A guide for practitioners, Studies in Fuzziness and Soft Computing 221 (2007).
[65] R. R. Yager, On ordered weighted averaging aggregation operators in multicriteria decisionmaking, IEEE Transactions on Systems, Man, and Cybernetics 18 (1) (1988) 183–190.
[66] G. H. Hardy, J. E. Littlewood, G. Pólya, Inequalities, 2nd Edition, Cambridge University Press, 1952.
[67] U.S. Department of Defense, MIL-STD-498: Software development and documentation, Military standard, U.S. Department of Defense (1994).
[68] R. C. Martin, Agile Software Development: Principles, Patterns, and Practices, Prentice Hall, 2003.
[69] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.
[70] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: Proc. 25th Int. Conf. on Machine Learning (ICML), 2008, pp. 1192–1199.
[71] Team SimPy, Simpy: Event discrete simulation for Python, `https://simpy.readthedocs.io` (2020).
[72] International Organization for Standardization, ISO/IEC 25022:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of quality in use,

Tech. rep., International Organization for Standardization (2016).
[73] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, T. Azumi, Autoware on board: Enabling autonomous vehicles with embedded systems, in: Proc. ACM/IEEE 9th Int. Conf. on Cyber-Physical Systems (ICCPS), 2018, pp. 287–296.
[74] Google Cloud Platform, Online boutique: A cloud-native microservices demo application, Software artifact (2024).
[75] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, IEEE Transactions on Software Engineering 47 (2) (2021) 243–260.
[76] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.
[77] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.
[78] C. Spearman, The proof and measurement of association between two things, American Journal of Psychology 15 (1) (1904) 72–101.
[79] M. D. Morris, Factorial sampling plans for preliminary computational experiments, Technometrics 33 (2) (1991) 161–174.
[80] F. Campolongo, J. Cariboni, A. Saltelli, An effective screening design for sensitivity analysis of large models, Environmental Modelling & Software 22 (10) (2007) 1509–1518.
[81] A. Saltelli, M. Ratto, T. Andres, F. Campolongo, J. Cariboni, D. Gatelli, M. Saisana, S. Tarantola, Global Sensitivity Analysis: The Primer, John Wiley & Sons, 2008.
[82] I. M. Sobol', Sensitivity estimates for nonlinear mathematical models, Mathematical Modelling and Computational Experiments 1 (4) (1993) 407–414.
[83] [REPLACE WITH FULL AUTHOR LIST], Software-as-a-graph: replication package (datasets, generator configurations, simulation harnesses, model checkpoints and analysis scripts), `https://doi.org/XX.XXXX/zenodo.XXXXXXX`, [dataset] PLACEHOLDER – insert version and DOI before submission (2026).